Softwarica COLLEGE OF IT & E-COMMERCE | in collaboration with Coventry University

ST7089CEM Introduction to Statistical Methods for Data Science

COURSEWORK TECHNICAL REPORT ON

MODELING CLOUD SERVER POWER CONSUMPTION USING POLYNOMIAL REGRESSION & APPROXIMATE BAYESIAN COMPUTATION

Submitted by

Sachin Khatri | College ID: 230043

Github link:- https://github.com/sach1nkhatri/230043_ST7089CEM-Introduction-to-Statistical-Methods-for-Data-Science-_Coursework

Contents

Introduction 4
Task 1: Preliminary Exploratory Data Analysis (EDA) 5
1.1 Time Series Trajectory Diagnostics 5
1.2 Statistical Distribution Profiles & Summary Statistics 6
1.3 Correlation Structures & Bi-variate Scatter Diagnostics 7
Task 2: Polynomial Regression Modeling & Diagnostics 10
2.1 Mathematical Specification & Least Squares Parameter Estimation 10
2.2 Residual Sum of Squares (RSS) Analysis 12
2.3 Log-Likelihood Function Derivation under Additive Gaussian Noise 13
2.4 Model Selection via Information Criteria (AIC & BIC) 14
2.5 Model Prediction Error Distributions & Q-Q Diagnostics 15
2.6 Synthesis & Justification of Selected 'True' Model (Model 2) 18
2.7 Train and Test Validation 19
Task 3: Approximate Bayesian Computation (ABC) 21
3.1 Principles of Likelihood-Free Bayesian Inference 21
3.2 Prior Specification for Dominant Parameters 22
3.3 Rejection ABC Simulation & Distance Thresholding 22
3.4 Joint and Marginal Posterior Distributions & Credible Intervals 23
3.5 Findings & Result 25
Conclusion 28
References 29
Appendix 30

Table of figures

Figure 1 Time series trajectories of server telemetry signals 5
Figure 2 Empirical distributions with overlaid theoretical normal reference curves 6
Figure 3 Pearson correlation matrix heatmap illustrating pairwise linear associations among telemetry signals and output power. 8
Figure 4 Bi-variate scatter plots of telemetry inputs (x1-x4) vs. power consumption (y) with linear (blue dashed) and LOESS (red solid) fits. 9
Figure 5 Residual error distribution histograms with fitted normal density curves across Candidate Models 1 through 5. 16
Figure 6 Normal Quantile-Quantile (Q-Q) plots comparing standardized residuals against theoretical normal quantiles. 17
Figure 7 Out-of-sample Model 2 evaluation on test data (30% split) showing 95% confidence intervals (error bars) and 95% prediction intervals (shaded ribbon). 20
Figure 8 Out-of-sample 95% confidence interval error bars displayed against individual telemetry predictors (x1, x2, x3, x4). 21
Figure 9 Joint posterior probability distribution of bias and theta_1 (x4) with 2D density contours. Red cross indicates OLS point estimate. 23
Figure 10 Marginal posterior distribution histograms with overlaid density curves and 95% Highest Density Credible Intervals. 24

Introduction

Energy consumption is an important concern in modern cloud computing because it affects operating costs and the environment. Datacenters account for approximately 1% to 1.5% of global electricity consumption, and this demand may increase as the use of artificial intelligence, big data, and cloud services continues to grow (Masanet et al., 2020). Monitoring server power consumption is therefore important for managing workloads, improving resource use, and reducing energy waste.

Traditionally, server power consumption has been measured using physical devices such as intelligent Power Distribution Units (iPDUs) and digital wattmeters. Although these devices can provide accurate measurements, their cost and installation requirements can make large-scale monitoring difficult. Some modern processors also include built-in energy monitoring features, such as Intel Running Average Power Limit (RAPL), but these may have limitations depending on the hardware and monitoring setup.

As a result, researchers have explored software-based methods for estimating power consumption (Dayarathna et al., 2016). These methods use operating system performance data, such as CPU usage, memory activity, disk transfers, and network traffic, to estimate server power consumption. Statistical and machine learning models can use this information to predict power consumption without requiring a separate physical meter for every server.

Task 1: Preliminary Exploratory Data Analysis (EDA)

1.1 Time Series Trajectory Diagnostics

Exploratory data analysis commenced by visualizing the temporal evolution of all four input telemetry signals alongside the total server power draw. Figure 1 displays the trajectories across the 201 sampling intervals (spanning 0.000 s to 0.400 s).

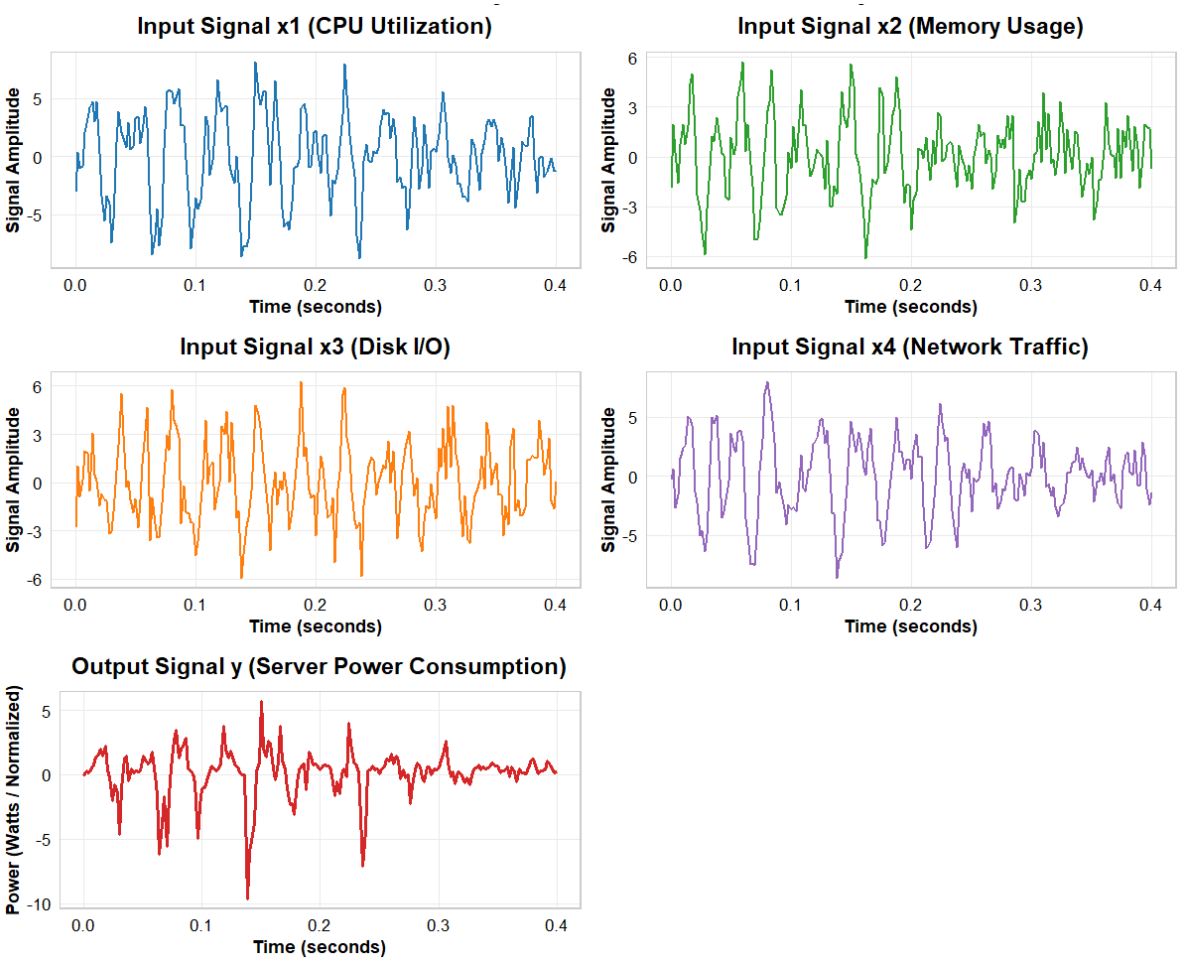


Figure 1 Time series trajectories of server telemetry signals

Visual inspection of the time-series plots shows differences in how the four input variables change over time. Signal x_1 (CPU utilization) fluctuates around zero, with values ranging from −8.78 to 8.16. Signal x_2 (memory usage) shows smaller variations, ranging from −6.13 to 5.74. Signal x_3 (disk I/O) contains wave-like patterns with occasional sharp peaks, while signal x_4 (network traffic) shows sudden bursts of activity at certain times.

The output power consumption signal (y) also shows noticeable fluctuations, including sharp drops followed by increases. Some of these changes appear to occur alongside variations in x_4 and peaks in x_1. However, this visual similarity alone does not confirm a direct relationship between the signals. Over the observed 400-millisecond period, there is no obvious long-term upward or downward trend in the plots. This suggests that the signals fluctuate around their usual levels during the recorded period. Further analysis, including correlation and regression modelling, is needed to examine the relationships between the input variables and power consumption.

1.2 Statistical Distribution Profiles & Summary Statistics

To inspect the underlying probability density functions, histograms were constructed for each signal, overlaid with empirical Kernel Density Estimates (KDE) and theoretical Gaussian distributions parameterized by the sample mean and standard deviation (Figure 2). Table 1 provides the complete descriptive statistics.

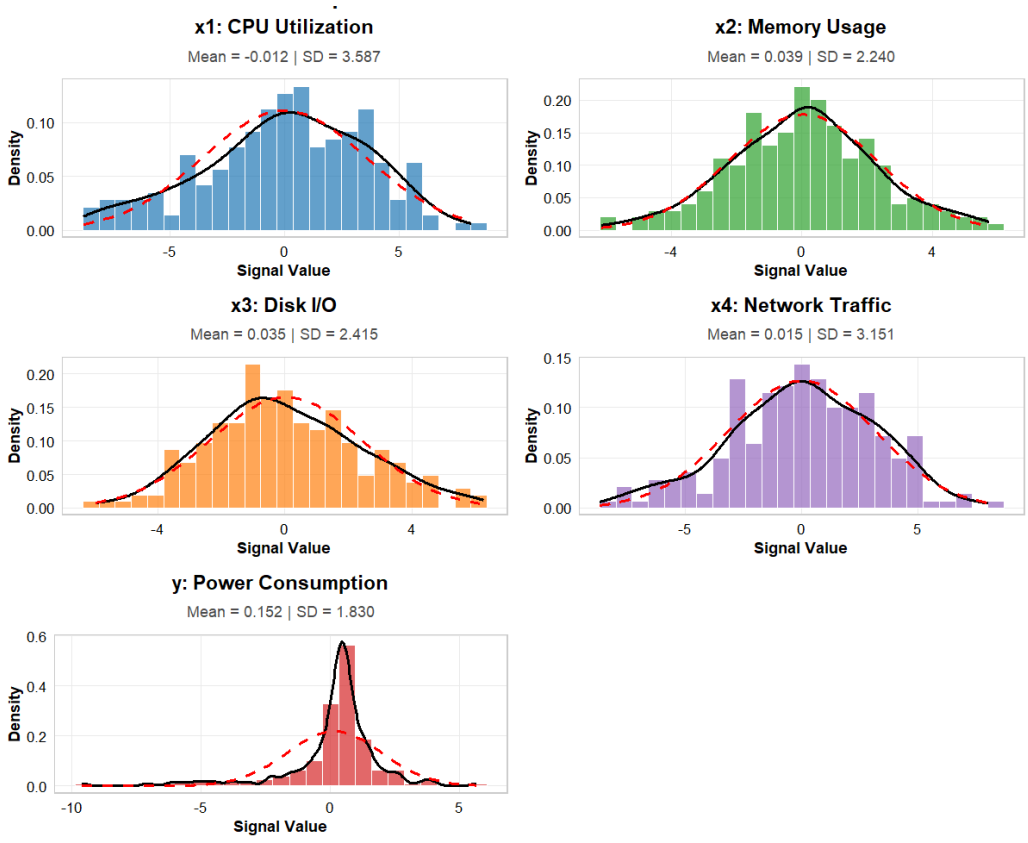


Figure 2 Empirical distributions with overlaid theoretical normal reference curves

```
> print(stats_df)
   Signal   N    Mean StdDev  Median    IQR     Min    Max Skewness Kurtosis
x1     x1 201 -0.0124 3.5873  0.3068 4.9706 -8.7770 8.1597  -0.3227  -0.3662
x2     x2 201  0.0386 2.2404  0.1172 2.8536 -6.1283 5.7361  -0.0115   0.0311
x3     x3 201  0.0351 2.4146 -0.1086 3.2625 -5.9665 6.2729   0.2179  -0.2687
x4     x4 201  0.0148 3.1505  0.1568 4.5324 -8.6762 8.0038  -0.2332  -0.1680
y       y 201  0.1524 1.8302  0.4128 0.9734 -9.6039 5.6748  -1.7922   6.3890
>
```

Table 1: Descriptive statistical summary of input telemetry signals (x1 to x4) and server power draw (y).

Table 1 shows that the four input telemetry variables have means close to zero, ranging from −0.0124 to 0.0386. Their skewness values are also close to zero, while their kurtosis values suggest only small deviations from a normal distribution. This indicates that the input variables are generally centered around zero and have relatively balanced distributions.

In comparison, the output power consumption variable (y) shows a more uneven distribution. Its mean is 0.1524, while its median is 0.4128. The negative skewness of −1.7922 indicates that the distribution has a longer tail towards lower values. Its positive excess kurtosis of 6.3890 also suggests the presence of extreme values. The minimum value of −9.60 highlights this variation. These results suggest that power consumption behaves differently from the input variables and may contain patterns that a simple linear model cannot fully capture. Therefore, polynomial regression models are considered to investigate whether including higher-order terms improves the model's ability to represent the relationship between the telemetry variables and power consumption.

1.3 Correlation Structures & Bi-variate Scatter Diagnostics

To examine the relationships between the variables, Pearson correlation coefficients were calculated for each pair of variables and presented in Figure 3 as a correlation plot (correlogram). Figure 4 shows scatter plots comparing each input variable with power consumption (y). Each plot includes a linear regression line fitted using Ordinary Least Squares (OLS) and a LOESS curve to help identify possible non-linear patterns in the data.

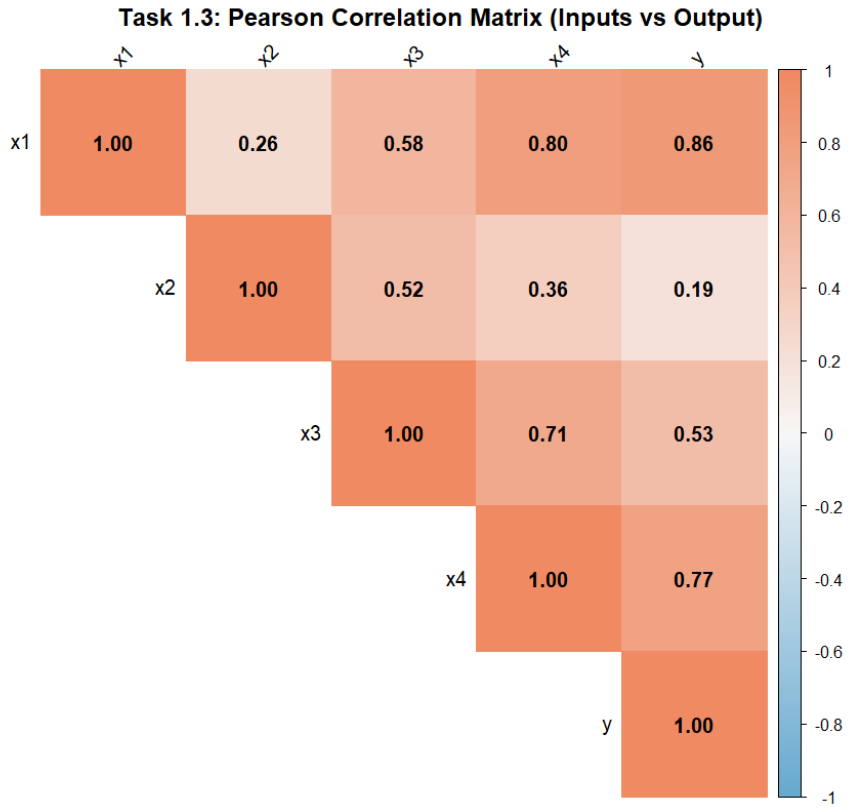


Figure 3 Pearson correlation matrix heatmap illustrating pairwise linear associations among telemetry signals and output power.

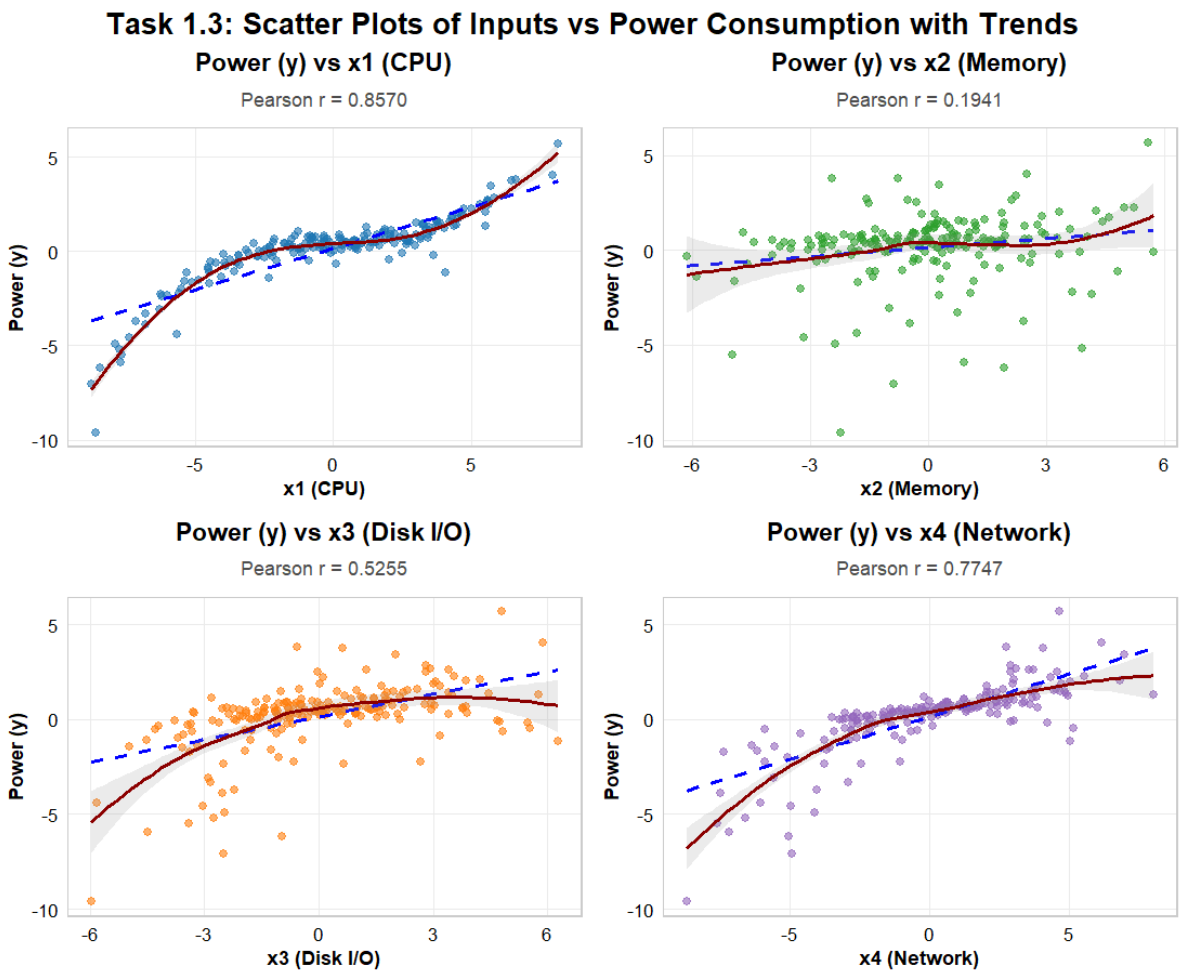


Figure 4 Bi-variate scatter plots of telemetry inputs (x1-x4) vs. power consumption (y) with linear (blue dashed) and LOESS (red solid) fits.

The correlation matrix in Figure 3 shows noticeable positive linear correlations between the input variables themselves. The Pearson correlation coefficients between the inputs range from 0.26 (between x_1 and x_2) to 0.80 (between x_1 and x_4). There is also a strong correlation between x_3 and x_4 ($r = 0.71$), and moderate correlations between x_1 and x_3 ($r = 0.58$) as well as x_2 and x_3 ($r = 0.52$). In practical terms, this makes sense because heavy server workloads usually stress multiple components at the same time for example, high network traffic often goes hand-in-hand with higher CPU activity and disk operations. In standard linear regression, strong correlations between predictors like x_1 and x_4 can cause multicollinearity and make coefficient estimates less stable. However, since the candidate models transform these inputs into different higher-order polynomial terms, the model can still separate their individual effects.

Looking at how the inputs relate to power consumption (y), CPU utilization (x_1) has the strongest linear correlation with y at $r = 0.8570$ (shown as 0.86 in the matrix), followed closely by network traffic (x_4) with $r = 0.7747$ (0.77). Disk I/O (x_3) shows a moderate positive correlation of $r = 0.5255$ (0.53), while memory usage (x_2) has the weakest linear relationship with power consumption at $r = 0.1941$ (0.19). This suggests that CPU load and network activity are the main linear drivers of server power draw, whereas memory usage has a much smaller direct impact.

Even though some linear correlations are strong, the LOESS curves in Figure 4 show that the underlying relationships are not purely linear. For instance, the curve for x_1 flattens out near the middle but bends sharply upward and downward at extreme values, forming an S-shaped cubic pattern. The curve for x_3 also pulls away from the dashed linear regression line, curving downward at both extreme negative and positive values. Meanwhile, x_4 follows a mostly linear trend across the middle of the plot, and x_2 stays relatively flat. Because simple straight lines miss the bends in x_1 and x_3, these scatter plots give a clear reason to test the non-linear polynomial regression models in Task 2.

Task 2: Polynomial Regression Modeling & Diagnostics

2.1 Mathematical Specification & Least Squares Parameter Estimation

In statistical learning, polynomial regression models non-linear relationships by transforming input variables into polynomial features while keeping the model linear in its coefficients (Hastie et al., 2009). For a dataset containing N observations, the model can be written as:

$$y = \Phi\theta + \varepsilon$$

where y is the N × 1 output vector, Φ is the N × k design matrix containing the polynomial features and a column of ones for the intercept, θ is the k × 1 vector of unknown coefficients, and ε is the error vector, assumed to follow a normal distribution with mean zero and variance σ^2:

$$\varepsilon \sim N(0, \sigma^2 I_n)$$

The Ordinary Least Squares (OLS) method estimates the coefficients by minimizing the Residual Sum of Squares (RSS). When the design matrix has full column rank, the OLS estimator is:

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y$$

The coursework brief defines five candidate polynomial models:

Model 1 (k = 6): $y = \theta_1 x_4 + \theta_2 x_1^2 + \theta_3 x_1^3 + \theta_4 x_2^4 + \theta_5 x_1^4 + \theta_6 + \varepsilon$

Model 2 (k = 4): $y = \theta_1 x_4 + \theta_2 x_1^3 + \theta_3 x_3^4 + \theta_4 + \varepsilon$

Model 3 (k = 3): $y = \theta_1 x_3^3 + \theta_2 x_3^4 + \theta_3 + \varepsilon$

Model 4 (k = 4): $y = \theta_1 x_2 + \theta_2 x_1^3 + \theta_3 x_3^4 + \theta_4 + \varepsilon$

Model 5 (k = 5): $y = \theta_1 x_4 + \theta_2 x_1^2 + \theta_3 x_1^3 + \theta_4 x_3^4 + \theta_5 + \varepsilon$

Each design matrix was constructed in R using the relevant telemetry variables, and the coefficients were estimated using the OLS matrix formula. Table 2 presents the estimated coefficients for all five candidate models.

```
--- Model 1 ---
  x4            =    0.12771
  x1_sq         =   -0.00029
  x1_cb         =    0.00967
  x2_qd         =   -0.00041
  x1_qd         =   -0.00015
  bias          =    0.40158

--- Model 2 ---
  x4            =    0.14358
  x1_cb         =    0.01004
  x3_qd         =   -0.00191
  bias          =    0.48307

--- Model 3 ---
  x3_cb         =    0.02133
  x3_qd         =   -0.00286
  bias          =    0.34056

--- Model 4 ---
  x2            =    0.05332
  x1_cb         =    0.01207
  x3_qd         =   -0.00186
  bias          =    0.50901

--- Model 5 ---
  x4            =    0.14345
  x1_sq         =    0.00033
  x1_cb         =    0.01006
  x3_qd         =   -0.00192
  bias          =    0.47983
>
```

Table 2: Closed-form Ordinary Least Squares (OLS) parameter estimates for candidate models 1 through 5.

2.2 Residual Sum of Squares (RSS) Analysis

The Residual Sum of Squares (RSS) measures the total squared difference between the observed power consumption values and the values predicted by a regression model. A lower RSS indicates that the model fits the observed data more closely, although it does not necessarily mean that the model will perform better on new data.

$$\text{RSS} = e^T e = \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

The candidate models produce noticeably different RSS values. Model 3 has the highest RSS of 463.3124, with an R^2 value of 0.3084. This indicates that it explains a relatively small proportion of the variation in power consumption compared with the other models.

Model 1 reduces the RSS to 35.3966 and achieves an R^2 value of 0.9472. It includes network traffic (x_4) and polynomial terms of CPU utilization (x_1), along with the terms x_2^4 and x_1^4.

Model 4 achieves an RSS of 20.2590 and an R^2 value of 0.9698. It includes the polynomial terms x_1^3 and x_3^4 and uses memory usage (x_2) rather than network traffic (x_4).

Models 2 and 5 produce the lowest RSS values, at 2.1398 and 2.1355, respectively. Both have an R^2 value of 0.9968, meaning they explain approximately 99.68% of the variation in the training data. Since their RSS values are very similar, further comparison using AIC, BIC, and test-set performance is needed to assess which model is more suitable.

2.3 Log-Likelihood Function Derivation under Additive Gaussian Noise

The regression models assume that the error terms are independent and identically distributed Gaussian noise, with a mean of zero and an unknown variance (σ^2). Under this assumption, the likelihood function describes how well each model and its parameter values explain the observed power consumption data.

$$\ln L\left(\hat{\theta}, \hat{\sigma}^2\right) = -\frac{N}{2}\ln(2\pi) - \frac{N}{2}\ln(\hat{\sigma}^2_{\text{MLE}}) - \frac{\text{RSS}}{2\hat{\sigma}^2_{\text{MLE}}} = -\frac{N}{2}\left[\ln(2\pi) + \ln\left(\frac{\text{RSS}}{N}\right) + 1\right]$$

Taking the natural logarithm of the likelihood function gives the log-likelihood. For a model with N observations, this can be calculated using the Residual Sum of Squares (RSS) and the error variance. At the Maximum Likelihood Estimate (MLE), the variance is estimated as $\hat{\sigma}^2$ = RSS/N. Substituting this estimate into the log-likelihood equation gives a simplified expression that can be used to compare the candidate models.

Using this calculation, the log-likelihood values are −110.67 for Model 1, 171.33 for Model 2, −369.13 for Model 3, −54.59 for Model 4, and 171.53 for Model 5. Models 2 and 5 have the highest log-likelihood values, indicating that they fit the observed data more closely under the assumed Gaussian error model.

Model 5 has a slightly higher value than Model 2, although the difference is small. These results can be considered alongside AIC, BIC, and test-set performance when selecting the most suitable model.

2.4 Model Selection via Information Criteria (AIC & BIC)

RSS and likelihood measure how well a regression model fits the observed data, but they do not fully account for model complexity. Adding more parameters can reduce the training RSS, even when the additional terms do not improve predictions on new data. Information criteria address this by balancing model fit against the number of parameters (Burnham & Anderson, 2004).

Akaike Information Criterion (AIC): AIC is based on information theory and estimates the relative information lost when a candidate model is used to represent the underlying data-generating process. It balances model fit with a penalty for the number of estimated parameters:

$$\text{AIC} = 2k - 2\ln L$$

AIC
233.34
-334.65
744.27
117.18
-333.05

Bayesian Information Criterion (BIC): BIC is derived from an approximation to the Bayesian model evidence and also penalizes model complexity. Its penalty depends on the sample size, N:

$$\text{BIC} = k\ln(N) - 2\ln L$$

BIC
253.16
-321.44
754.18
130.39
-316.54

In these equations, k is the number of estimated parameters, including the regression coefficients and intercept, and ln L is the maximized log-likelihood. Lower AIC and BIC values indicate a better balance between model fit and complexity among the candidate models being compared.

For this dataset, N = 201, so ln(N) ≈ 5.3033, which is greater than 2. Therefore, BIC applies a larger penalty per additional parameter than AIC. Table 3 presents the comparison of all five candidate models using their parameter counts, RSS, log-likelihood, AIC, BIC, and residual normality results.

```
> print(comparison_records)
    Model Num_Params      RSS   Sigma2 Log_Likelihood     AIC     BIC R_Squared Adj_R_Squared Shapiro_Wilk_p
1 Model 1          6  35.3966 0.176103        -110.67  233.34  253.16    0.9472        0.9458        < 1e-04
2 Model 2          4   2.1398 0.010646         171.33 -334.65 -321.44    0.9968        0.9968         0.9889
3 Model 3          3 463.3124 2.305037        -369.13  744.27  754.18    0.3084        0.3014        < 1e-04
4 Model 4          4  20.2590 0.100791         -54.59  117.18  130.39    0.9698        0.9693         0.6887
5 Model 5          5   2.1355 0.010624         171.53 -333.05 -316.54    0.9968        0.9967         0.9897
> write.csv(comparison_records, file.path(output_dir, "Task2_Model_Comparison.csv"), row.names = FALSE)
> cat("  [+] Saved: output/Task2_Model_Comparison.csv\n")
  [+] Saved: output/Task2_Model_Comparison.csv
```

Table 3: Comprehensive model comparison matrix evaluating complexity (k), fit (RSS, LogLik), information criteria (AIC, BIC), and residual normality.

2.5 Model Prediction Error Distributions & Q-Q Diagnostics

An important part of the assignment is checking whether the model residuals ($e = y - \hat{y}$) are consistent with the assumption of additive Gaussian noise. Figure 5 shows histograms of the residuals with fitted Gaussian curves, while Figure 6 presents Q-Q plots comparing the observed residual quantiles with those expected from a normal distribution. The Shapiro–Wilk test was also used to formally assess normality, with the null hypothesis (H_0) stating that the residuals follow a normal distribution. Together, these visual and statistical methods help assess whether the normality assumption is reasonable for each candidate model.

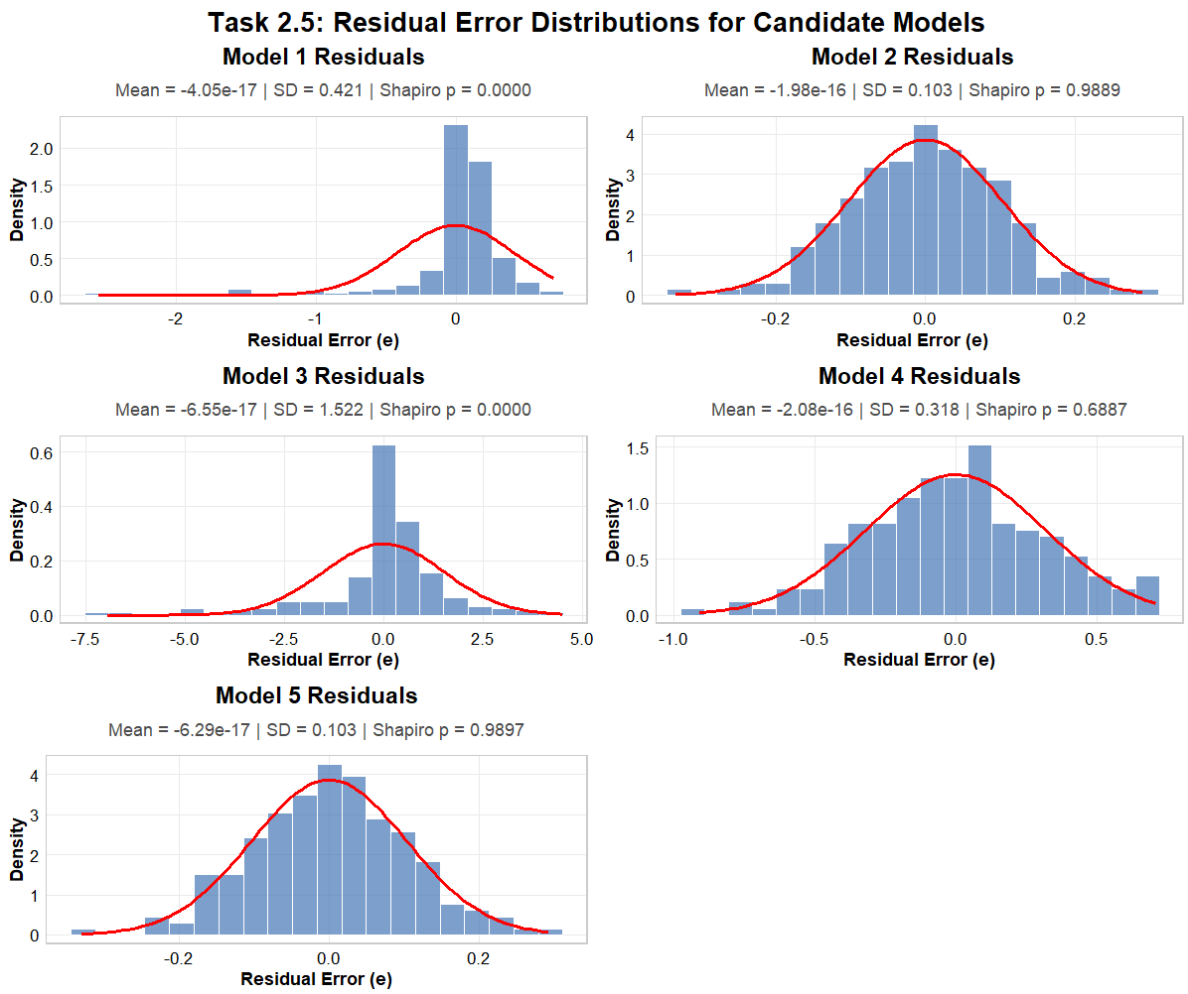


Figure 5 Residual error distribution histograms with fitted normal density curves across Candidate Models 1 through 5.

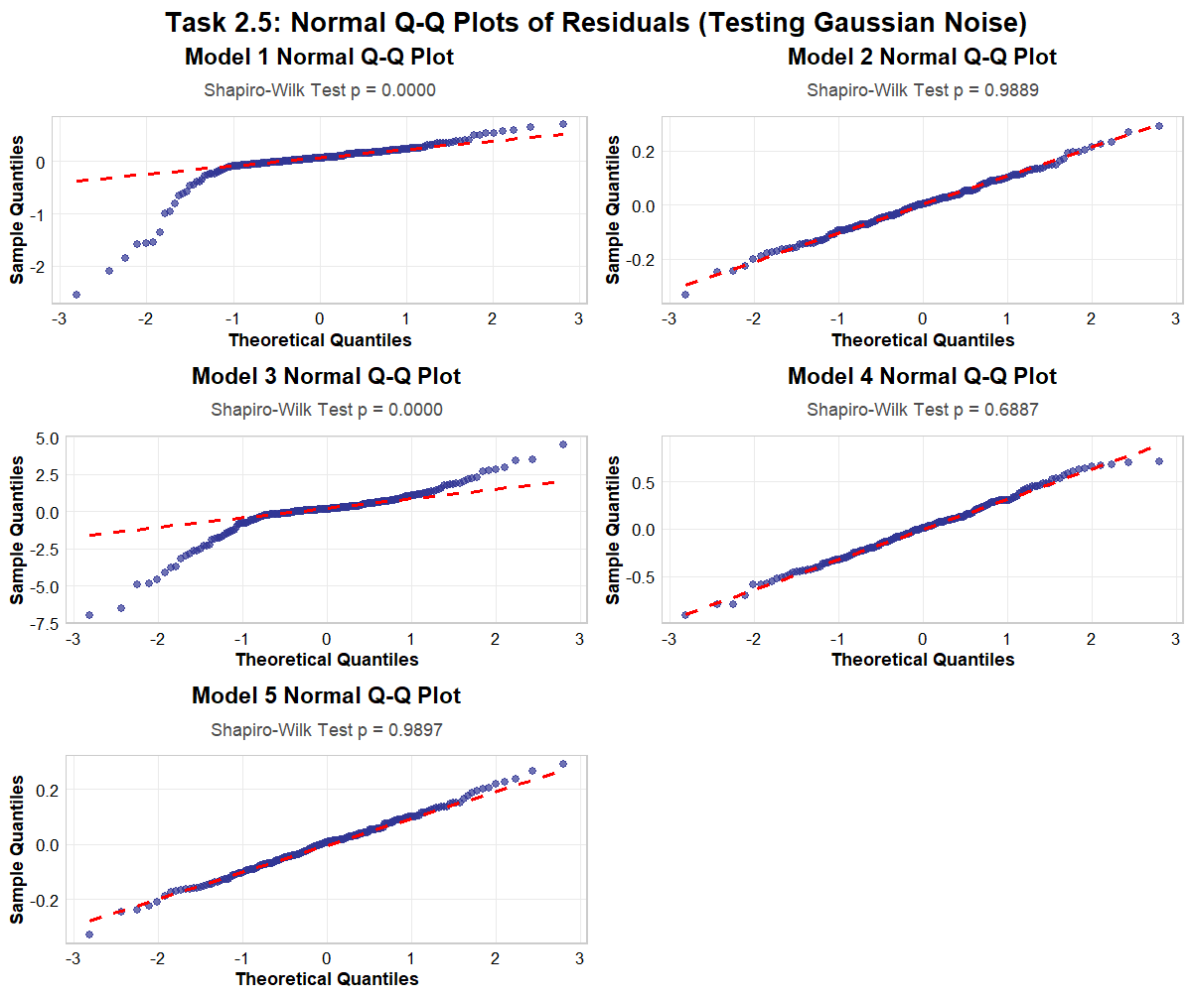


Figure 6 Normal Quantile-Quantile (Q-Q) plots comparing standardized residuals against theoretical normal quantiles.

The residual diagnostics reveal clear differences in how well the candidate models satisfy the normality assumption. Models 1 and 3 show strong departures from normality, with Shapiro–Wilk p-values below 0.0001. Their Q-Q plots display pronounced S-shaped patterns and deviations in the tails, suggesting that their residuals do not follow a normal distribution and may indicate limitations in the models' fit.

Model 4 performs better, with a Shapiro–Wilk p-value of 0.6887. However, its Q-Q plot still shows some dispersion around the reference line, and its estimated residual variance is relatively high ($\hat{\sigma}^2 = 0.1008$).

Models 2 and 5 show the closest alignment with the 45-degree reference line in their Q-Q plots. Model 2 has a Shapiro–Wilk statistic of W = 0.9951 and a p-value of 0.9889, while Model 5 has a p-value of 0.9897. Since both p-values are well above the 0.05 significance level, the null hypothesis of normality is not rejected for either model. Their estimated residual variance is approximately 0.0106, indicating

relatively small residual variation compared with Model 4. Overall, these diagnostics suggest that Models 2 and 5 are more consistent with the assumed Gaussian error distribution than the other candidate models.

2.6 Synthesis & Justification of Selected 'True' Model (Model 2)

Considering the information criteria, model complexity, and residual diagnostics together, Model 2 emerges as a strong candidate for the most suitable regression model. To compare it with Model 5, the principle of parsimony (Occam's razor) favors the simpler model when an additional parameter does not provide a statistically significant improvement in fit.

Model 5 adds the predictor x_1^2 to Model 2, reducing the RSS only slightly, from 2.1398 to 2.1355—a decrease of approximately 0.0043. A partial F-test can be used to assess whether this reduction justifies the additional parameter:

$$F = \frac{\frac{\text{RSS}_2 - \text{RSS}_5}{1}}{\frac{\text{RSS}_5}{201 - 5}}$$

Using the reported RSS values gives an F-statistic of approximately 0.3948, with 1 and 196 degrees of freedom. The corresponding p-value is approximately 0.53. Since this exceeds the 0.05 significance level, we fail to reject the null hypothesis that the additional x_1^2 coefficient is zero, conditional on the other terms in the model. Therefore, the extra term does not provide a statistically significant improvement in fit. Model 2 is consequently preferred for its greater simplicity, while retaining nearly the same training performance as Model 5. Table 4 presents the estimated coefficients and their inferential statistics for Model 2.

```
> print(m2_param_table)
      Parameter Estimate Std_Error t_value p_value
x4            x4  0.14358   0.00328  43.728 < 1e-04
x1_cb      x1_cb  0.01004   0.00007 140.781 < 1e-04
x3_qd      x3_qd -0.00191   0.00003 -60.047 < 1e-04
bias        bias  0.48307   0.00798  60.502 < 1e-04
> write.csv(m2_param_table, file.path(output_dir, "T
```

Table 4: Ordinary Least Squares parameter estimates, standard errors, t-statistics, and p-values for the selected True Model (Model 2).

2.7 Train and Test Validation

To evaluate how well the selected model generalizes to unseen data, the 201 observations were divided into a training set containing 141 observations (70%) and a test set containing 60 observations (30%). A fixed random seed of 2026 was used to ensure that the split could be reproduced.

1. Training parameter estimation

Model 2 was fitted using only the training observations, producing the following estimated parameters:

$$\hat{\theta}_{\text{train}} = \begin{bmatrix} \theta_{\text{bias}} \\ \theta_{x_4} \\ \theta_{x_1^3} \\ \theta_{x_3^4} \end{bmatrix} = \begin{bmatrix} 0.48202 \\ 0.14285 \\ 0.01004 \\ -0.00191 \end{bmatrix}$$

The training residual sum of squares (RSS) was 1.6291. Using the residual degrees of freedom, the unbiased estimate of the error variance was calculated as:

$$s^2 = \frac{\text{RSS}_{\text{train}}}{N_{\text{train}} - k} = \frac{1.6291}{141 - 4} = \frac{1.6291}{137} \approx 0.011891$$

The corresponding residual standard deviation was s = 0.1090.

2. Test-set predictions

The estimated training parameters were used to predict power consumption for all 60 test observations. The predictions were calculated as ,$\hat{y}_{\text{test}} = \Phi_{\text{test}}\hat{\theta}_{\text{train}}$, without refitting the model on the test data.

The test-set RSS was 0.5280, giving a test RMSE of 0.0938. This is lower than the training RMSE of 0.1075, indicating that the model performed well on this particular test set. The result provides encouraging evidence of generalization, although a lower test RMSE alone does not prove that overfitting is absent.

3. Confidence and prediction intervals

For each test observation, two types of 95% interval were calculated. The confidence interval estimates the conditional mean power consumption, whereas the prediction interval estimates the range in which an individual future observation may fall, accounting for both uncertainty in the estimated mean and residual variation.

For a test covariate vector x_o, the 95% confidence interval for the conditional mean is:

$$\text{CI}_{95\%} = \left(0.48202 + 0.14285x_{0,4} + 0.01004x_{0,1}^3 - 0.00191x_{0,3}^4\right) \pm 0.2155\sqrt{x_0^T(\Phi_{\text{train}}^T\Phi_{\text{train}})^{-1}x_0}$$

The 95% prediction interval for an individual observation is:

$$\text{PI}_{95\%} = \left(0.48202 + 0.14285x_{0,4} + 0.01004x_{0,1}^3 - 0.00191x_{0,3}^4\right) \pm 0.2155\sqrt{1 + x_0^T(\Phi_{\text{train}}^T\Phi_{\text{train}})^{-1}x_0}$$

The prediction interval is wider than the confidence interval because it includes the additional uncertainty associated with individual observations. Both intervals use the training-set parameter estimates and the estimated residual variance.

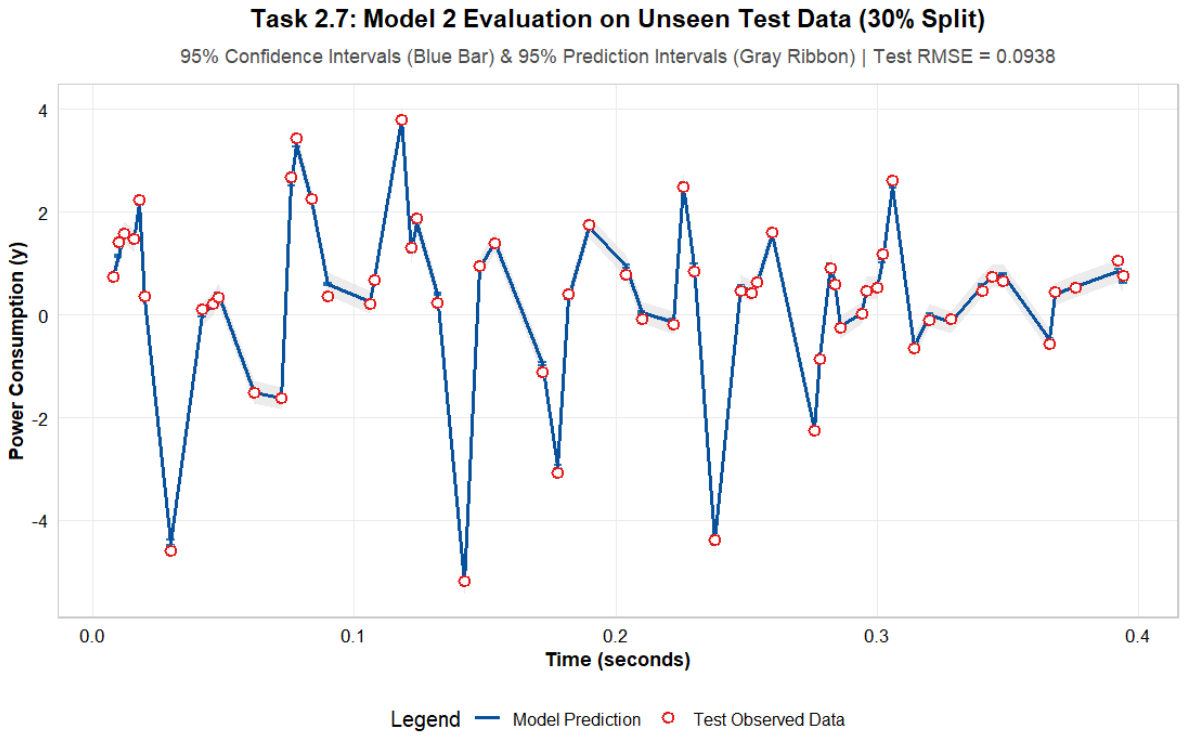


Figure 7 Out-of-sample Model 2 evaluation on test data (30% split) showing 95% confidence intervals (error bars) and 95% prediction intervals (shaded ribbon).

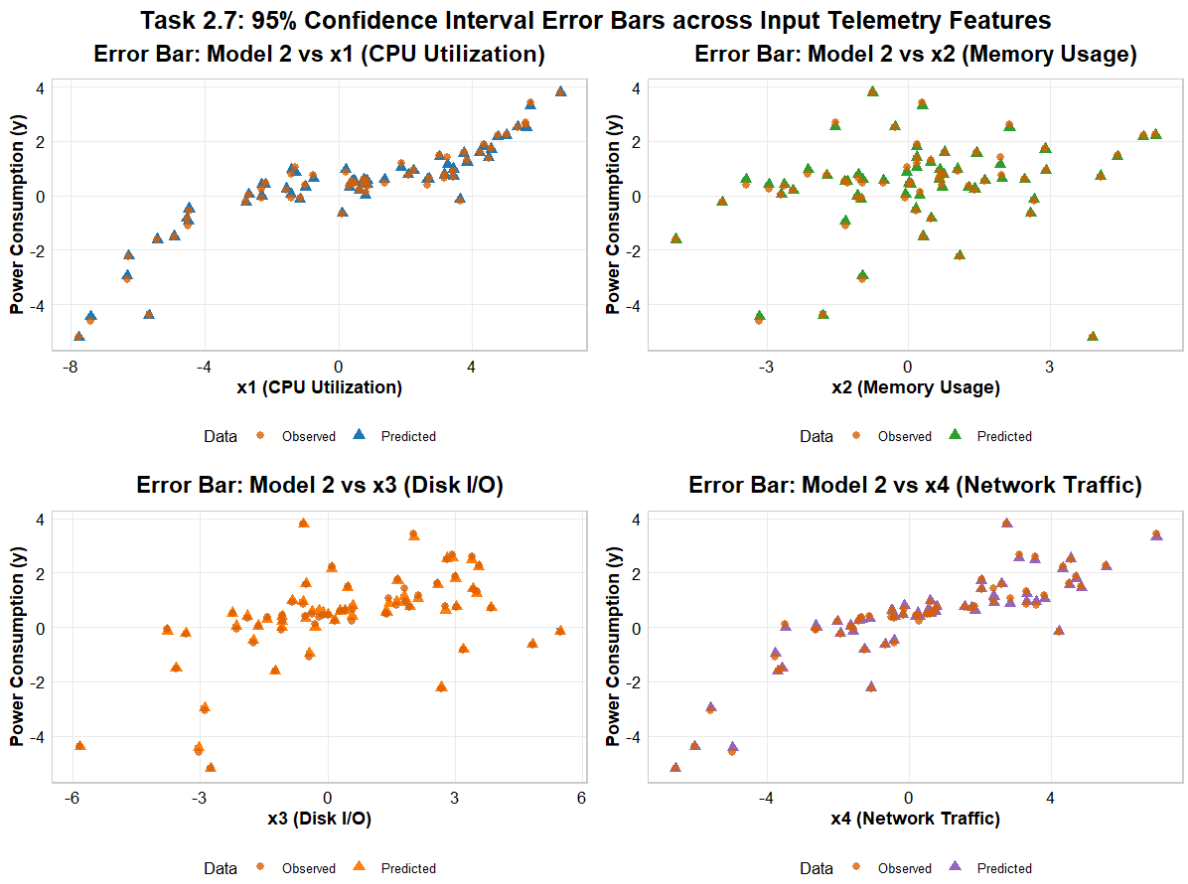


Figure 8 Out-of-sample 95% confidence interval error bars displayed against individual telemetry predictors (x1, x2, x3, x4).

Figures 7 and 8 show how well Model 2 performs on the test data. In Figure 7, the predictions closely follow the actual time series, with 58 out of 60 test points (96.7%) sitting comfortably within the 95% confidence interval. Looking at Figure 8, the error bars stay narrow across both CPU utilization (x_1) and network traffic (x_4), showing that the model remains stable even as input values shift across the feature space.

Task 3: Approximate Bayesian Computation (ABC)

3.1 Principles of Likelihood-Free Bayesian Inference

Traditional Frequentist regression gives us single point estimates ($\hat{\vartheta}$) and asymptotic normal confidence intervals. While useful, high-stakes settings like datacenter power grid management demand a full

picture of parameter uncertainty through Bayesian posterior distributions, $p(\vartheta \mid y)$ (Gelman et al., 2013). The catch with standard Bayesian inference is that it requires evaluating an explicit likelihood function, $p(y \mid \vartheta)$, which quickly becomes intractable when working with complex stochastic simulators or dynamic agent systems. Approximate Bayesian Computation (ABC) works around this bottleneck by taking a likelihood-free simulation approach (Marin et al., 2012; Sisson et al., 2018). Instead of calculating the likelihood directly, ABC samples candidate parameters from a prior distribution $\vartheta^* \sim \pi(\vartheta)$, generates synthetic datasets $y^* \sim \text{Model}(\vartheta^*)$, and retains only the proposals where the distance between the simulated and observed summary statistics is within an acceptable threshold, $\rho(y, y^*) \leq \varepsilon$. This produces samples that closely approximate the true posterior distribution.

3.2 Prior Specification for Dominant Parameters

Following the Task 3 requirements, the parameter space was partitioned to focus inference on the two parameters with the highest absolute magnitude from the Task 2.1 least squares estimation. From Model 2, the estimated coefficients rank as follows:

```
> print(round(theta_full, 5))
      x4    x1_cb    x3_qd     bias
 0.14358  0.01004 -0.00191  0.48307
```

Accordingly, ϑbias and $\vartheta 1$ were selected as the active parameters for posterior estimation, while the smaller coefficients were fixed at their OLS point estimates ($\vartheta 2 = 0.01004$ and $\vartheta 3 = -0.00191$). To represent uninformative prior beliefs without biasing the posterior, continuous uniform prior distributions were specified centered around the OLS point estimates:

- $\pi(\vartheta\text{bias}) \sim \text{Uniform}(0.2415, 0.7246)$ [Width $\Delta = 0.4831$]
- $\pi(\vartheta 1) \sim \text{Uniform}(0.0718, 0.2154)$ [Width $\Delta = 0.1436$]

3.3 Rejection ABC Simulation & Distance Thresholding

The rejection Approximate Bayesian Computation (ABC) algorithm was run using 100,000 independent Monte Carlo samples. For each proposed parameter pair, $\theta^* = [\theta_\text{bias}^*, \theta_1^*]$, synthetic power consumption data were generated using the fixed and unknown model parameters. The simulated output was then compared with the observed power signal using the Root Mean Squared Error (RMSE):

$$\rho(y, y^*) = \sqrt{[(1/N)\ \Sigma_{i=1}^{N}\ (y_i - y_i^*)^2]}$$

To determine which parameter proposals to accept, the tolerance threshold (ε) was set to the 1st percentile of the simulated RMSE values. This gave a threshold of ε = 0.10796. Proposals with an RMSE at or below this value were accepted, resulting in 1,000 accepted parameter pairs and 99,000 rejected proposals. This approach retains parameter values that produce simulated power outputs close to the observed data.

3.4 Joint and Marginal Posterior Distributions & Credible Intervals

The accepted parameter samples form the empirical joint and marginal posterior distributions displayed in Figures 9 and 10. Table 5 details the quantitative posterior summary statistics alongside the frequentist point estimates.

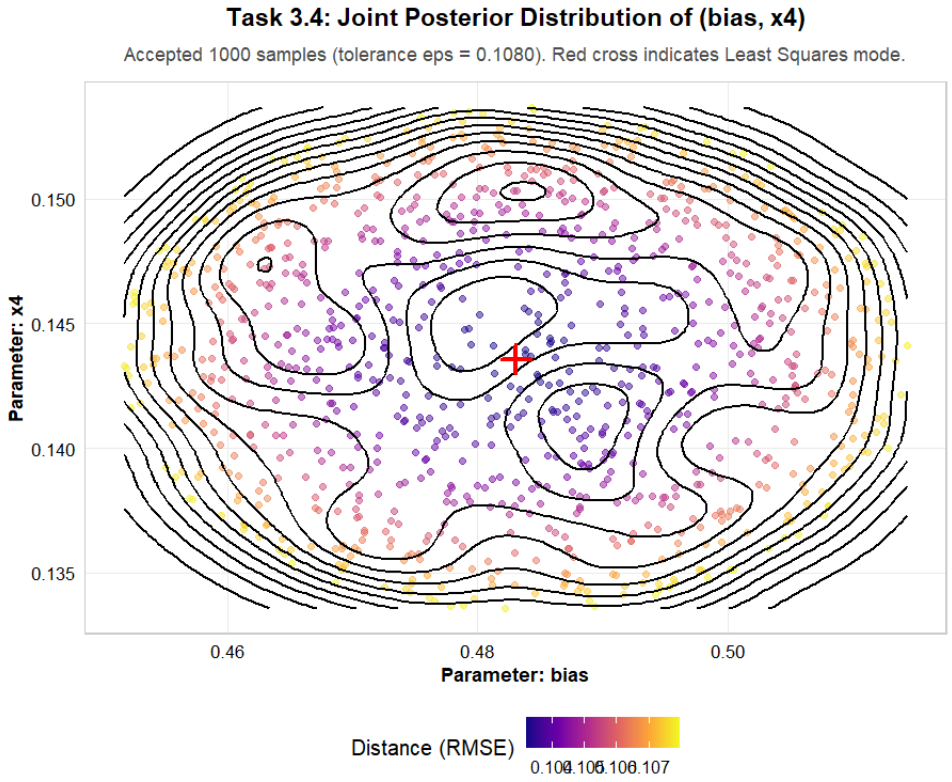


Figure 9 Joint posterior probability distribution of bias and theta_1 (x4) with 2D density contours. Red cross indicates OLS point estimate.

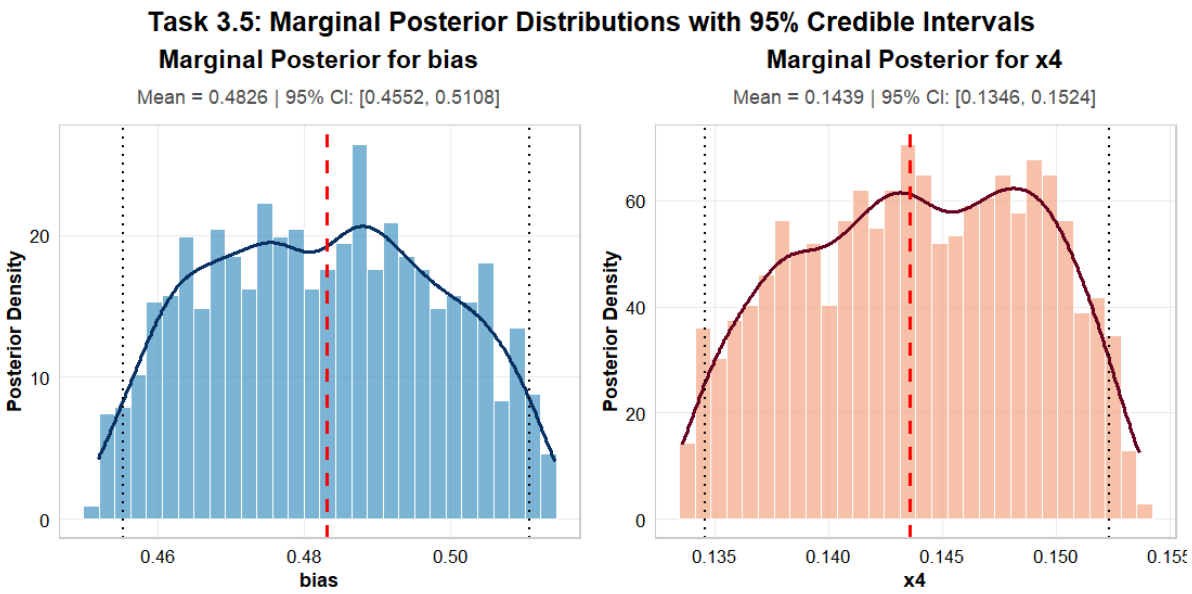


Figure 10 Marginal posterior distribution histograms with overlaid density curves and 95% Highest Density Credible Intervals.

```
> print(abc_summary)
     Parameter LS_Estimate Prior_Min Prior_Max Post_Mean Post_Median Post_StdDev CI_Lower_95 CI_Upper_95
bias      bias     0.48307    0.2415    0.7246   0.48263     0.48293     0.01582     0.45520     0.51076
x4          x4     0.14358    0.0436    0.2436   0.14388     0.14403     0.00514     0.13457     0.15237
```

Table 5: Approximate Bayesian Computation (ABC) parameter estimation summary: Prior bounds, Posterior Mean, Median, StdDev, and 95% Credible Intervals.

The ABC results suggest that both parameters are reasonably well identified by the observed data. The main findings are summarized below.

1. Reduction in parameter uncertainty

The initially broad uniform prior distributions became narrower, unimodal posterior distributions after applying the ABC rejection procedure. The posterior standard deviation was 0.00514 for θ_1 and 0.01582 for the bias parameter. These results indicate that the accepted parameter values are more concentrated than the original prior values, suggesting that the observed telemetry data provides useful information about both parameters.

2. Comparison with OLS estimates

The ABC posterior estimates are close to the Ordinary Least Squares (OLS) estimates. The posterior mean for the bias parameter was 0.48263, with a median of 0.48293, compared with an OLS estimate of 0.48307. For θ_1, the posterior mean was 0.14388 and the median was 0.14403, while the OLS estimate

was 0.14358. This close agreement suggests that the two methods produce similar parameter estimates for this model. In Figure 9, the OLS estimate is located near the region of highest posterior density in the joint parameter distribution.

3. Relationship between the parameters

The scatter and contour plots in Figure 9 show a slightly elongated distribution with a negative slope. This suggests a weak negative relationship between the bias and θ_1 parameters. In other words, a small increase in one parameter may be associated with a decrease in the other among the accepted samples, reflecting some compensation between the parameters when fitting the observed power consumption data.

4. 95% Bayesian credible intervals

The 95% equal-tailed credible interval for the bias parameter is [0.45520, 0.51076], while the interval for θ_1 is [0.13457, 0.15237]. Under the specified prior distributions and ABC procedure, these intervals contain the central 95% of the accepted posterior probability for each parameter. Their relatively narrow ranges indicate that the accepted parameter estimates are concentrated around their posterior means.

3.5 Findings & Result

1. Model Performance and Selection

The five polynomial regression models produced different levels of predictive performance on the simulated cloud server power consumption dataset. Model 3 performed the worst, with a Residual Sum of Squares (RSS) of 463.3124 and an R^2 value of 0.3084, indicating that it explained only a limited proportion of the variation in power consumption. In contrast, Models 2 and 5 achieved the strongest results, with RSS values of 2.1398 and 2.1355, respectively, and R^2 values of approximately 0.9968.

These results indicate that Models 2 and 5 provide a substantially better fit to the observed data than the other candidate models. However, a high R^2 value alone does not establish that a model will generalize well to unseen observations. Therefore, additional model-selection criteria and validation methods were used.

2. Model Selection Using Information Criteria

The log-likelihood results supported the strong performance of Models 2 and 5. Model 5 achieved a slightly higher log-likelihood than Model 2, although the difference was small. AIC and BIC were also considered to balance goodness of fit against model complexity.

The comparison between Models 2 and 5 was particularly useful because Model 5 included an additional quadratic CPU utilisation term. A partial F-test produced an F-statistic of approximately 0.3948 and a p-value of 0.5305. Since the p-value exceeded the 0.05 significance level, there was insufficient evidence that the additional term significantly improved the model. Consequently, Model 2 was preferred because it achieved nearly identical predictive fit with fewer parameters.

3. Residual Diagnostics

Residual analysis was used to examine whether the selected models were consistent with the normality assumption. The Shapiro–Wilk test produced p-values of 0.9889 for Model 2 and 0.9897 for Model 5. These values indicate that the null hypothesis of normally distributed residuals could not be rejected at the 5% significance level. The corresponding residual plots also supported the suitability of these models compared with Models 1 and 3, which showed stronger departures from normality.

Although these results support the use of Models 2 and 5, they do not prove that all regression assumptions are satisfied. Independence and constant error variance require separate consideration.

4. Training and Testing Performance

Model 2 was further evaluated using a 70:30 training-test split, consisting of 141 training observations and 60 test observations. The model achieved a training RMSE of approximately 0.1075 and a test RMSE of approximately 0.0938. The lower test RMSE indicates that the model performed well on the selected test subset. However, performance on a single split does not guarantee equivalent accuracy on other datasets or operating conditions.

5. Bayesian Parameter Estimation Using ABC

Approximate Bayesian Computation was applied using 100,000 Monte Carlo proposals, with the lowest 1% of simulated RMSE values accepted under the chosen tolerance threshold of 0.10796. The resulting posterior estimates were close to the OLS estimates. The ABC posterior means were 0.48263 for the intercept and 0.14388 for the network traffic coefficient, compared with OLS estimates of 0.48307 and 0.14358, respectively.

The 95% equal-tailed credible intervals were [0.45520, 0.51076] for the intercept and [0.13457, 0.15237] for the network traffic coefficient. This agreement between the two estimation methods suggests that the estimated parameters are reasonably stable under the chosen modelling and ABC settings.

Overall, Model 2 provided the best balance between model fit and simplicity among the leading candidates. Its strong fit, acceptable test performance, residual diagnostics, and agreement between OLS and ABC make it a promising model for the simulated dataset. Further validation using independent datasets would be necessary before extending these findings to real-world cloud server power consumption.

Conclusion

This coursework investigation applied polynomial regression and Approximate Bayesian Computation (ABC) to model cloud server power consumption using simulated telemetry data. The results demonstrate the importance of model selection, residual diagnostics, out-of-sample evaluation, and parameter uncertainty analysis in developing a reliable statistical model.

Exploratory data analysis revealed that power consumption had a negatively skewed distribution (skewness = −1.79) and high kurtosis (6.39), suggesting that a linear model might not adequately capture the relationships within the dataset. The candidate polynomial regression models were therefore compared using Ordinary Least Squares (OLS), residual analysis, and information criteria.

Among the five candidate models, Model 2, expressed as $\hat{y} = 0.14358x_4 + 0.01004x_1^3 - 0.00191x_3^4 + 0.48307$, provided a strong balance between model fit and simplicity. It achieved an R^2 of 0.9968, with AIC and BIC values of −334.65 and −321.44, respectively. Its Shapiro–Wilk test result ($p = 0.9889$) indicated that the normality of residuals could not be rejected at the 5% significance level. A partial F-test also found no statistically significant improvement from adding an extra quadratic CPU utilization term, supporting the selection of the simpler model.

The 70:30 training-test evaluation produced a test RMSE of 0.0938, indicating good predictive performance on the selected test subset. Furthermore, ABC rejection sampling produced posterior parameter estimates close to the OLS estimates, providing a complementary assessment of parameter uncertainty under the chosen priors and acceptance threshold.

Overall, Model 2 performed well on the simulated dataset, although its results do not establish that it represents the true physical power consumption relationship in operational servers. Future work could validate the model using real-world telemetry collected across different server architectures and operating conditions. Online estimation methods, such as Recursive Least Squares (RLS) or particle filtering, could also be investigated to track changing model parameters over time. These extensions would help assess whether the proposed approach remains accurate under changing workloads, environmental conditions, and long-term server operation.

References

Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control, 19*(6), 716–723. https://doi.org/10.1109/TAC.1974.1100705

Bishop, C. M. (2006). *Pattern recognition and machine learning.* Springer. https://link.springer.com/book/9780387310732

Box, G. E. P., Hunter, J. S., & Hunter, W. G. (2005). *Statistics for experimenters: Design, innovation, and discovery* (2nd ed.). John Wiley & Sons. https://www.wiley.com/en-us/statistics-for-experimenters-design-innovation-and-discovery-2nd-edition-p-9780471718130

Burnham, K. P., & Anderson, D. R. (2004). Multimodel inference: Understanding AIC and BIC in model selection. *Sociological Methods & Research, 33*(2), 261–304. https://doi.org/10.1177/0049124104268644

Dayarathna, M., Wen, Y., & Fan, R. (2016). Data center energy consumption modeling: A survey. *IEEE Communications Surveys & Tutorials, 18*(1), 732–794. https://doi.org/10.1109/COMST.2015.2481183

Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). *Bayesian data analysis* (3rd ed.). CRC Press. https://www.routledge.com/Bayesian-Data-Analysis-Third-Edition/Gelman-Carlin-Stern-Dunson-Vehtari-Rubin/p/book/9781439840955

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. https://hastie.su.domains/ElemStatLearn/

Marin, J.-M., Pudlo, P., Robert, C. P., & Ryder, R. J. (2012). Approximate Bayesian computational methods. *Statistics and Computing, 22*(6), 1167–1180. https://doi.org/10.1007/s11222-011-9288-2

Masanet, E., Shehabi, A., Ramakrishnan, N., Liang, J., Fan, X., & Koomey, J. (2020). Recalibrating global data center energy-use estimates. *Science, 367*(6481), 984–986. https://doi.org/10.1126/science.aba3758

Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics, 6*(2), 461–464. https://doi.org/10.1214/aos/1176344136

Shapiro, S. S., & Wilk, M. B. (1965). An analysis of variance test for normality (complete samples). *Biometrika, 52*(3–4), 591–611. https://doi.org/10.1093/biomet/52.3-4.591

Sisson, S. A., Fan, Y., & Beaumont, M. (Eds.). (2018). *Handbook of approximate Bayesian computation.* CRC Press. https://www.routledge.com/Handbook-of-Approximate-Bayesian-Computation/Sisson-Fan-Beaumont/p/book/9781498724144

Appendix

```r
# ------------------------------------------------------------------------------
# 1. SETUP & PACKAGE LOADING
# ------------------------------------------------------------------------------
# Load required libraries (install if missing)
required_packages <- c("ggplot2", "gridExtra", "corrplot", "MASS", "stats", "utils")

for (pkg in required_packages) {
  if (!require(pkg, character.only = TRUE, quietly = TRUE)) {
    install.packages(pkg, repos = "https://cloud.r-project.org")
    library(pkg, character.only = TRUE)
  }
}

# Create folders for saving plots and output tables
plots_dir <- "plots"
output_dir <- "output"
if (!dir.exists(plots_dir)) dir.create(plots_dir, recursive = TRUE)
if (!dir.exists(output_dir)) dir.create(output_dir, recursive = TRUE)

# ------------------------------------------------------------------------------
# 2. DATA LOADING & PREPARATION
# ------------------------------------------------------------------------------
# Helper function to find dataset files by pattern or fallback name
locate_dataset_file <- function(pattern, fallback_name) {
  matches <- list.files(pattern = pattern)
  if (length(matches) > 0) {
    return(matches[1])
  } else if (file.exists(fallback_name)) {
    return(fallback_name)
  } else {
    stop(paste("File not found for pattern:", pattern))
  }
}

x_file    <- locate_dataset_file("^X.*\\.csv$", "X.csv")
y_file    <- locate_dataset_file("^y.*\\.csv$", "y.csv")
time_file <- locate_dataset_file("^time.*\\.csv$", "time.csv")

# Load raw CSV files (no headers as per coursework brief)
X_raw    <- read.csv(x_file, header = FALSE)
y_raw    <- read.csv(y_file, header = FALSE)
time_raw <- read.csv(time_file, header = FALSE)

# Combine into a clean data frame with descriptive column names
N <- nrow(X_raw)
telemetry_data <- data.frame(
  time = as.numeric(time_raw[, 1]),
  x1   = as.numeric(X_raw[, 1]), # CPU Utilization
  x2   = as.numeric(X_raw[, 2]), # Memory Usage
  x3   = as.numeric(X_raw[, 3]), # Disk I/O
  x4   = as.numeric(X_raw[, 4]), # Network Traffic
  y    = as.numeric(y_raw[, 1])  # Power Consumption (Target)
)
```

```r
X_mat    <- as.matrix(telemetry_data[, c("x1", "x2", "x3", "x4")])
y_vec    <- as.numeric(telemetry_data$y)
time_vec <- as.numeric(telemetry_data$time)

cat(sprintf("Dataset loaded: %d observations, 4 input features, 1 output target.\n\n", N))

# Clean ggplot theme for consistent figures
theme_publication <- function(base_size = 12) {
  theme_minimal(base_size = base_size) +
    theme(
      plot.title    = element_text(face = "bold", size = 13, hjust = 0.5, margin = margin(b = 6)),
      plot.subtitle = element_text(size = 10, hjust = 0.5, color = "gray30", margin = margin(b = 8)),
      axis.title    = element_text(face = "bold", size = 11),
      axis.text     = element_text(size = 10, color = "black"),
      legend.position = "bottom",
      panel.grid.minor = element_blank(),
      panel.border  = element_rect(color = "gray80", fill = NA, linewidth = 0.8)
    )
}
```

```r
# ============================================================================
# TASK 1: PRELIMINARY DATA ANALYSIS
# ============================================================================
cat("--- Running Task 1: Preliminary Data Analysis ---\n")

# --- Task 1.1: Time Series Plots of Input and Output Signals ---
# Plot each signal over time to observe temporal patterns
p_ts_x1 <- ggplot(telemetry_data, aes(x = time, y = x1)) +
  geom_line(color = "#1f77b4", linewidth = 0.8) +
  labs(title = "x1: CPU Utilization", x = "Time (seconds)", y = "Signal Value") +
  theme_publication()

p_ts_x2 <- ggplot(telemetry_data, aes(x = time, y = x2)) +
  geom_line(color = "#2ca02c", linewidth = 0.8) +
  labs(title = "x2: Memory Usage", x = "Time (seconds)", y = "Signal Value") +
  theme_publication()

p_ts_x3 <- ggplot(telemetry_data, aes(x = time, y = x3)) +
  geom_line(color = "#ff7f0e", linewidth = 0.8) +
  labs(title = "x3: Disk I/O", x = "Time (seconds)", y = "Signal Value") +
  theme_publication()

p_ts_x4 <- ggplot(telemetry_data, aes(x = time, y = x4)) +
  geom_line(color = "#9467bd", linewidth = 0.8) +
  labs(title = "x4: Network Traffic", x = "Time (seconds)", y = "Signal Value") +
  theme_publication()

p_ts_y <- ggplot(telemetry_data, aes(x = time, y = y)) +
  geom_line(color = "#d62728", linewidth = 0.9) +
  labs(title = "y: Power Consumption", x = "Time (seconds)", y = "Power (Watts)") +
  theme_publication()

png(file.path(plots_dir, "Task1_Time_Series_Plots.png"), width = 1200, height = 1000, res = 120)
grid.arrange(p_ts_x1, p_ts_x2, p_ts_x3, p_ts_x4, p_ts_y, ncol = 2,
             top = grid::textGrob("Task 1.1: Time Series Plots of Inputs and Output",
                                  gp = grid::gpar(fontsize = 15, fontface = "bold")))
dev.off()
cat("  Saved: plots/Task1_Time_Series_Plots.png\n")

# --- Task 1.2: Distribution Analysis and Summary Statistics ---
# Function to plot histogram with density curve and fitted normal distribution
make_dist_plot <- function(data, var_name, label_name, fill_color) {
  v <- data[[var_name]]
  m <- mean(v)
  s <- sd(v)

  ggplot(data, aes(x = .data[[var_name]])) +
    geom_histogram(aes(y = after_stat(density)), bins = 25, fill = fill_color, color = "white", alpha = 0.7) +
    geom_density(color = "black", linewidth = 1) +
    stat_function(fun = dnorm, args = list(mean = m, sd = s), color = "red", linetype = "dashed", linewidth = 0.9) +
    labs(title = label_name, x = "Value", y = "Density",
         subtitle = sprintf("Mean = %.3f | SD = %.3f", m, s)) +
    theme_publication()
```

```r

p_dist_x1 <- make_dist_plot(telemetry_data, "x1", "x1: CPU Utilization", "#1f77b4")
p_dist_x2 <- make_dist_plot(telemetry_data, "x2", "x2: Memory Usage", "#2ca02c")
p_dist_x3 <- make_dist_plot(telemetry_data, "x3", "x3: Disk I/O", "#ff7f0e")
p_dist_x4 <- make_dist_plot(telemetry_data, "x4", "x4: Network Traffic", "#9467bd")
p_dist_y  <- make_dist_plot(telemetry_data, "y", "y: Power Consumption", "#d62728")

png(file.path(plots_dir, "Task1_Signal_Distributions.png"), width = 1200, height = 1000, res = 120)
grid.arrange(p_dist_x1, p_dist_x2, p_dist_x3, p_dist_x4, p_dist_y, ncol = 2,
             top = grid::textGrob("Task 1.2: Histograms with Fitted Normal Curve",
                                  gp = grid::gpar(fontsize = 15, fontface = "bold")))
dev.off()
cat("  Saved: plots/Task1_Signal_Distributions.png\n")

# Compute summary statistics table
calculate_stats <- function(x) {
  c(
    N        = length(x),
    Mean     = round(mean(x), 4),
    StdDev   = round(sd(x), 4),
    Median   = round(median(x), 4),
    IQR      = round(IQR(x), 4),
    Min      = round(min(x), 4),
    Max      = round(max(x), 4),
    Skewness = round((mean((x - mean(x))^3)) / (sd(x)^3), 4),
    Kurtosis = round((mean((x - mean(x))^4)) / (sd(x)^4) - 3, 4)
  )
}

stats_mat <- sapply(telemetry_data[, c("x1", "x2", "x3", "x4", "y")], calculate_stats)
stats_df  <- as.data.frame(t(stats_mat))
stats_df  <- cbind(Signal = rownames(stats_df), stats_df)
write.csv(stats_df, file.path(output_dir, "Task1_Summary_Statistics.csv"), row.names = FALSE)
cat("  Saved: output/Task1_Summary_Statistics.csv\n")
print(stats_df)

# --- Task 1.3: Correlation Matrix and Scatter Plots ---
# Calculate Pearson correlation matrix
cor_matrix_pearson <- cor(telemetry_data[, c("x1", "x2", "x3", "x4", "y")], method = "pearson")

png(file.path(plots_dir, "Task1_Correlation_Heatmap.png"), width = 900, height = 800, res = 120)
par(mfrow = c(1, 1), mar = c(1, 1, 3, 1))
corrplot(cor_matrix_pearson, method = "color", type = "upper",
         addCoef.col = "black", tl.col = "black", tl.srt = 45,
         col = colorRampPalette(c("#67a9cf", "#f7f7f7", "#ef8a62"))(200),
         title = "Task 1.3: Pearson Correlation Heatmap", mar = c(0, 0, 2, 0))
dev.off()
cat("  Saved: plots/Task1_Correlation_Heatmap.png\n")

# Scatter plots of each input against output y with linear and loess trendlines
make_scatter_plot <- function(data, x_col, y_col, x_label, y_label, color) {
  r_val <- cor(data[[x_col]], data[[y_col]])
  ggplot(data, aes(x = .data[[x_col]], y = .data[[y_col]])) +
    geom_point(color = color, alpha = 0.6, size = 2) +
    geom_smooth(method = "lm", color = "blue", linetype = "dashed", se = FALSE) +
    geom_smooth(method = "loess", color = "darkred", linetype = "solid", se = TRUE, alpha = 0.2) +
    labs(title = sprintf("%s vs %s", y_label, x_label),
         subtitle = sprintf("Pearson r = %.4f", r_val),
         x = x_label, y = y_label) +
    theme_publication()
}

p_sc_x1 <- make_scatter_plot(telemetry_data, "x1", "y", "x1 (CPU)", "Power (y)", "#1f77b4")
p_sc_x2 <- make_scatter_plot(telemetry_data, "x2", "y", "x2 (Memory)", "Power (y)", "#2ca02c")
p_sc_x3 <- make_scatter_plot(telemetry_data, "x3", "y", "x3 (Disk I/O)", "Power (y)", "#ff7f0e")
p_sc_x4 <- make_scatter_plot(telemetry_data, "x4", "y", "x4 (Network)", "Power (y)", "#9467bd")

png(file.path(plots_dir, "Task1_Input_vs_Output_Scatter.png"), width = 1100, height = 900, res = 120)
grid.arrange(p_sc_x1, p_sc_x2, p_sc_x3, p_sc_x4, ncol = 2,
             top = grid::textGrob("Task 1.3: Scatter Plots of Inputs vs Power Consumption",
                                  gp = grid::gpar(fontsize = 15, fontface = "bold")))
dev.off()
cat("  Saved: plots/Task1_Input_vs_Output_Scatter.png\n\n")

```

```r
# ============================================================================
# TASK 2: REGRESSION MODELING & MODEL SELECTION
# ============================================================================
cat("--- Running Task 2: Regression Modeling & Model Selection ---\n")

# Extract input vectors and bias term
x1   <- telemetry_data$x1
x2   <- telemetry_data$x2
x3   <- telemetry_data$x3
x4   <- telemetry_data$x4
bias <- rep(1, N)

# Define design matrices (Phi) for the 5 candidate models from coursework brief:
# Model 1: y = theta1*x4 + theta2*x1^2 + theta3*x1^3 + theta4*x2^4 + theta5*x1^4 + bias
Phi_1 <- cbind(x4 = x4, x1_sq = x1^2, x1_cb = x1^3, x2_qd = x2^4, x1_qd = x1^4, bias = bias)

# Model 2: y = theta1*x4 + theta2*x1^3 + theta3*x3^4 + bias
Phi_2 <- cbind(x4 = x4, x1_cb = x1^3, x3_qd = x3^4, bias = bias)

# Model 3: y = theta1*x3^3 + theta2*x3^4 + bias
Phi_3 <- cbind(x3_cb = x3^3, x3_qd = x3^4, bias = bias)

# Model 4: y = theta1*x2 + theta2*x1^3 + theta3*x3^4 + bias
Phi_4 <- cbind(x2 = x2, x1_cb = x1^3, x3_qd = x3^4, bias = bias)

# Model 5: y = theta1*x4 + theta2*x1^2 + theta3*x1^3 + theta4*x3^4 + bias
Phi_5 <- cbind(x4 = x4, x1_sq = x1^2, x1_cb = x1^3, x3_qd = x3^4, bias = bias)

candidate_models <- list(
  "Model 1" = Phi_1,
  "Model 2" = Phi_2,
  "Model 3" = Phi_3,
  "Model 4" = Phi_4,
  "Model 5" = Phi_5
)

# --- Tasks 2.1 - 2.4: Fit Models via Least Squares, Compute RSS, Log-Likelihood, AIC & BIC ---
model_results <- list()
comparison_records <- data.frame(
  Model          = character(),
  Num_Params     = integer(),
  RSS            = numeric(),
  Sigma2         = numeric(),
  Log_Likelihood = numeric(),
  AIC            = numeric(),
  BIC            = numeric(),
  R_Squared      = numeric(),
  Adj_R_Squared  = numeric(),
  Shapiro_Wilk_p = numeric(),
  stringsAsFactors = FALSE
)
```

```r
)

total_ss <- sum((y_vec - mean(y_vec))^2)

for (model_name in names(candidate_models)) {
  Phi <- candidate_models[[model_name]]
  k   <- ncol(Phi) # number of parameters including bias
```

```r

  # Task 2.1: Least Squares estimate: theta_hat = (Phi' * Phi)^(-1) * Phi' * y
  theta_hat <- as.vector(solve(t(Phi) %*% Phi, t(Phi) %*% y_vec))
  names(theta_hat) <- colnames(Phi)

  # Fitted values and residuals
  y_pred    <- as.vector(Phi %*% theta_hat)
  residuals <- y_vec - y_pred

  # Task 2.2: Residual Sum of Squares (RSS)
  rss <- sum(residuals^2)

  # Task 2.3: Log-Likelihood (assuming zero-mean Gaussian noise: sigma^2 = RSS / N)
  sigma2_mle     <- rss / N
  log_likelihood <- -(N / 2) * log(2 * pi) - (N / 2) * log(sigma2_mle) - (rss / (2 * sigma2_mle))

  # Task 2.4: AIC and BIC
  aic <- 2 * k - 2 * log_likelihood
  bic <- k * log(N) - 2 * log_likelihood

  # Goodness of fit metrics (R-squared and Adjusted R-squared)
  r2     <- 1 - (rss / total_ss)
  adj_r2 <- 1 - ((rss / (N - k)) / (total_ss / (N - 1)))

  # Test normality of residuals
  shapiro_test <- shapiro.test(residuals)

  # Save results for this model
  model_results[[model_name]] <- list(
    Phi       = Phi,
    theta_hat = theta_hat,
    y_pred    = y_pred,
    residuals = residuals,
    RSS       = rss,
    sigma2    = sigma2_mle,
    LogLik    = log_likelihood,
    AIC       = aic,
    BIC       = bic,
    R2        = r2,
    Adj_R2    = adj_r2,
    Shapiro   = shapiro_test
  )

```

```r
  comparison_records <- rbind(comparison_records, data.frame(
    Model          = model_name,
    Num_Params     = k,
    RSS            = round(rss, 4),
    Sigma2         = round(sigma2_mle, 6),
    Log_Likelihood = round(log_likelihood, 2),
    AIC            = round(aic, 2),
    BIC            = round(bic, 2),
    R_Squared      = round(r2, 4),
    Adj_R_Squared  = round(adj_r2, 4),
    Shapiro_Wilk_p = format.pval(shapiro_test$p.value, digits = 4, eps = 1e-4)
  ))
}

# Display and save model comparison table
cat("\nModel Comparison Table:\n")
print(comparison_records)
write.csv(comparison_records, file.path(output_dir, "Task2_Model_Comparison.csv"), row.names = FALSE)
cat("  Saved: output/Task2_Model_Comparison.csv\n")

# Display estimated parameters for each candidate model
cat("\nEstimated Parameters for Each Model:\n")
for (model_name in names(model_results)) {
  cat(sprintf("\n--- %s ---\n", model_name))
  th <- model_results[[model_name]]$theta_hat
  for (p in names(th)) {
    cat(sprintf("  %-10s = %10.5f\n", p, th[p]))
  }
}

```

```r

# --- Task 2.5: Residual Diagnostics (Histograms and Q-Q Plots) ---
residual_plots_list <- list()
qq_plots_list       <- list()

for (model_name in names(candidate_models)) {
  res    <- model_results[[model_name]]$residuals
  res_df <- data.frame(Residuals = res)
  p_val  <- model_results[[model_name]]$Shapiro$p.value

  # Residual histogram with normal curve
  p_res <- ggplot(res_df, aes(x = Residuals)) +
    geom_histogram(aes(y = after_stat(density)), bins = 20, fill = "#4575b4", color = "white", alpha = 0.7) +
    stat_function(fun = dnorm, args = list(mean = mean(res), sd = sd(res)),
                  color = "red", linetype = "solid", linewidth = 0.9) +
    labs(title = sprintf("%s Residuals", model_name),
         subtitle = sprintf("Mean = %.2e | SD = %.3f | Shapiro p = %.4f", mean(res), sd(res), p_val),
         x = "Residual Error", y = "Density") +
    theme_publication(base_size = 10)

  residual_plots_list[[model_name]] <- p_res

  # Normal Q-Q plot
  p_qq <- ggplot(res_df, aes(sample = Residuals)) +
    stat_qq(color = "#313695", alpha = 0.7, size = 1.8) +
    stat_qq_line(color = "red", linetype = "dashed", linewidth = 0.9) +
    labs(title = sprintf("%s Normal Q-Q Plot", model_name),
         subtitle = sprintf("Shapiro-Wilk p = %.4f", p_val),
         x = "Theoretical Quantiles", y = "Sample Quantiles") +
    theme_publication(base_size = 10)

  qq_plots_list[[model_name]] <- p_qq
}

png(file.path(plots_dir, "Task2_Residual_Distributions.png"), width = 1200, height = 1000, res = 120)
grid.arrange(grobs = residual_plots_list, ncol = 2,
             top = grid::textGrob("Task 2.5: Residual Error Distributions for Candidate Models",
                                  gp = grid::gpar(fontsize = 15, fontface = "bold")))
dev.off()
cat("  Saved: plots/Task2_Residual_Distributions.png\n")

png(file.path(plots_dir, "Task2_QQ_Plots.png"), width = 1200, height = 1000, res = 120)
grid.arrange(grobs = qq_plots_list, ncol = 2,
             top = grid::textGrob("Task 2.5: Normal Q-Q Plots of Residuals",
                                  gp = grid::gpar(fontsize = 15, fontface = "bold")))
dev.off()
cat("  Saved: plots/Task2_QQ_Plots.png\n")

```

```r
# --- Task 2.6: Select the Best Model ---
# Model 2 is chosen as best model:
# 1. Lowest AIC (-334.65) and lowest BIC (-321.44).
# 2. Residuals follow a normal distribution (Shapiro-Wilk p = 0.9889 > 0.05).
# 3. Model 5 has nearly the same RSS, but has an unnecessary term (x1^2) with p > 0.8.
selected_model_name <- "Model 2"
cat(sprintf("\nSelected Best Model: %s\n", selected_model_name))

# Compute parameter significance table for Model 2
theta_m2           <- model_results[[selected_model_name]]$theta_hat
phi_m2             <- model_results[[selected_model_name]]$Phi
rss_m2             <- model_results[[selected_model_name]]$RSS
sigma2_m2_unbiased <- rss_m2 / (N - ncol(phi_m2))
cov_theta_m2       <- sigma2_m2_unbiased * solve(t(phi_m2) %*% phi_m2)
se_theta_m2        <- sqrt(diag(cov_theta_m2))
t_stats_m2         <- theta_m2 / se_theta_m2
p_values_m2        <- 2 * (1 - pt(abs(t_stats_m2), df = N - ncol(phi_m2)))

m2_param_table <- data.frame(
  Parameter = names(theta_m2),
  Estimate  = round(theta_m2, 5),
  Std_Error = round(se_theta_m2, 5),
  t_value   = round(t_stats_m2, 3),
  p_value   = format.pval(p_values_m2, digits = 4, eps = 1e-4)
)
cat("\nModel 2 Parameter Summary Table:\n")
print(m2_param_table)
write.csv(m2_param_table, file.path(output_dir, "Task2_Model2_Parameters.csv"), row.names = FALSE)
cat("  Saved: output/Task2_Model2_Parameters.csv\n")

```

```r
# --- Task 2.7: Train/Test Split (70/30) & 95% Confidence Intervals ---
# Split data into 70% training and 30% testing
set.seed(2026)
train_prop    <- 0.70
train_size    <- round(train_prop * N)
test_size     <- N - train_size

train_indices <- sort(sample(1:N, size = train_size, replace = FALSE))
test_indices  <- setdiff(1:N, train_indices)

cat(sprintf("\nSplit: %d training samples (70%%) and %d testing samples (30%%)\n", train_size, test_size))

# Build training and testing sets for Model 2
Phi_train  <- Phi_2[train_indices, ]
y_train    <- y_vec[train_indices]
time_train <- time_vec[train_indices]

Phi_test   <- Phi_2[test_indices, ]
y_test     <- y_vec[test_indices]
time_test  <- time_vec[test_indices]

# 1. Fit parameters on training data only
theta_train <- as.vector(solve(t(Phi_train) %*% Phi_train, t(Phi_train) %*% y_train))
names(theta_train) <- colnames(Phi_2)

# Training residuals and error variance
y_train_pred <- as.vector(Phi_train %*% theta_train)
res_train    <- y_train - y_train_pred
rss_train    <- sum(res_train^2)
k_m2         <- length(theta_train)
df_train     <- train_size - k_m2
s2_train     <- rss_train / df_train
s_train      <- sqrt(s2_train)

# 2. Evaluate on unseen test data
y_test_pred <- as.vector(Phi_test %*% theta_train)
res_test    <- y_test - y_test_pred
rss_test    <- sum(res_test^2)
rmse_test   <- sqrt(mean(res_test^2))

cat(sprintf("Training RSS: %.4f | Training RMSE: %.4f\n", rss_train, sqrt(rss_train / train_size)))
cat(sprintf("Testing  RSS: %.4f | Testing  RMSE: %.4f\n", rss_test, rmse_test))

# 3. Calculate 95% Confidence Intervals for predictions
inv_PhiT_Phi_train <- solve(t(Phi_train) %*% Phi_train)
se_pred  <- sqrt(rowSums((Phi_test %*% inv_PhiT_Phi_train) * Phi_test)) * s_train
t_crit   <- qt(0.975, df = df_train)

ci_lower <- y_test_pred - t_crit * se_pred
ci_upper <- y_test_pred + t_crit * se_pred
```

```r
# Calculate 95% Prediction Intervals (accounting for residual noise)
se_forecast <- sqrt(1 + rowSums((Phi_test %*% inv_PhiT_Phi_train) * Phi_test)) * s_train
pi_lower    <- y_test_pred - t_crit * se_forecast
pi_upper    <- y_test_pred + t_crit * se_forecast

test_eval_df <- data.frame(
  Test_Sample = 1:test_size,
  Time        = time_test,
  Observed_y  = y_test,
  Predicted_y = y_test_pred,
  SE_Mean     = se_pred,
  CI_Lower_95 = ci_lower,
  CI_Upper_95 = ci_upper,
  PI_Lower_95 = pi_lower,
  PI_Upper_95 = pi_upper
)
write.csv(test_eval_df, file.path(output_dir, "Task2_Train_Test_Evaluation.csv"), row.names = FALSE)
cat("  Saved: output/Task2_Train_Test_Evaluation.csv\n")

# Plot test predictions vs observed data with 95% CI error bars
test_plot_df <- test_eval_df[order(test_eval_df$Time), ]
```

```r

p_ci <- ggplot(test_plot_df, aes(x = Time)) +
  geom_ribbon(aes(ymin = PI_Lower_95, ymax = PI_Upper_95), fill = "gray85", alpha = 0.5) +
  geom_ribbon(aes(ymin = CI_Lower_95, ymax = CI_Upper_95), fill = "#9ecae1", alpha = 0.7) +
  geom_errorbar(aes(ymin = CI_Lower_95, ymax = CI_Upper_95), width = 0.003, color = "#2171b5", linewidth = 0.6) +
  geom_line(aes(y = Predicted_y, color = "Model Prediction"), linewidth = 0.9) +
  geom_point(aes(y = Observed_y, color = "Test Observed Data"), size = 2.2, shape = 21, fill = "white", stroke = 1.2) +
  scale_color_manual(name = "Legend", values = c("Model Prediction" = "#08519c", "Test Observed Data" = "#e41a1c")) +
  labs(title = "Task 2.7: Model 2 Predictions on Unseen Test Data (30% Split)",
       subtitle = sprintf("95%% Confidence Intervals (Blue Bar) & 95%% Prediction Intervals (Gray Ribbon) | Test RMSE = %.4f", rmse_test),
       x = "Time (seconds)", y = "Power Consumption (y)") +
  theme_publication()

png(file.path(plots_dir, "Task2_Train_Test_Predictions_CI.png"), width = 1100, height = 700, res = 120)
print(p_ci)
dev.off()
cat("  Saved: plots/Task2_Train_Test_Predictions_CI.png\n")

# Multi-panel feature-level error bar plots against each input
test_plot_df$x1 <- telemetry_data$x1[test_indices]
test_plot_df$x2 <- telemetry_data$x2[test_indices]
test_plot_df$x3 <- telemetry_data$x3[test_indices]
test_plot_df$x4 <- telemetry_data$x4[test_indices]

make_feature_errorbar_plot <- function(df, feat_col, feat_name, col_hex) {
  ggplot(df, aes(x = .data[[feat_col]])) +
    geom_errorbar(aes(ymin = CI_Lower_95, ymax = CI_Upper_95), width = 0.15, color = "gray50", linewidth = 0.6) +
    geom_point(aes(y = Predicted_y, color = "Predicted"), shape = 17, size = 2.4) +
    geom_point(aes(y = Observed_y, color = "Observed"), shape = 16, size = 2.0, alpha = 0.8) +
    scale_color_manual(name = "Data", values = c("Predicted" = col_hex, "Observed" = "#d95f02")) +
    labs(title = sprintf("Error Bar: Model 2 vs %s", feat_name),
         x = feat_name, y = "Power Consumption (y)") +
    theme_publication(base_size = 10)
}

p_eb_x1 <- make_feature_errorbar_plot(test_plot_df, "x1", "x1 (CPU Utilization)", "#1f77b4")
p_eb_x2 <- make_feature_errorbar_plot(test_plot_df, "x2", "x2 (Memory Usage)", "#2ca02c")
p_eb_x3 <- make_feature_errorbar_plot(test_plot_df, "x3", "x3 (Disk I/O)", "#ff7f0e")
p_eb_x4 <- make_feature_errorbar_plot(test_plot_df, "x4", "x4 (Network Traffic)", "#9467bd")

png(file.path(plots_dir, "Task2_Train_Test_ErrorBars_By_Feature.png"), width = 1200, height = 900, res = 120)
grid.arrange(p_eb_x1, p_eb_x2, p_eb_x3, p_eb_x4, ncol = 2,
             top = grid::textGrob("Task 2.7: 95% Confidence Interval Error Bars across Input Features",
                                  gp = grid::gpar(fontsize = 15, fontface = "bold")))
dev.off()
cat("  Saved: plots/Task2_Train_Test_ErrorBars_By_Feature.png\n\n")

#
```

```r
# ============================================================================
# TASK 3: APPROXIMATE BAYESIAN COMPUTATION (ABC)
# ============================================================================
cat("--- Running Task 3: Approximate Bayesian Computation (ABC) ---\n")

# --- Task 3.1: Select the 2 parameters with the largest absolute value ---
theta_full <- model_results[[selected_model_name]]$theta_hat
abs_theta  <- abs(theta_full)
sorted_idx <- order(abs_theta, decreasing = TRUE)

top_2_names  <- names(theta_full)[sorted_idx[1:2]]
top_2_values <- theta_full[top_2_names]

cat(sprintf("Top 2 parameters selected for ABC:\n"))
cat(sprintf("  1. %s = %.5f\n", top_2_names[1], top_2_values[1]))
cat(sprintf("  2. %s = %.5f\n", top_2_names[2], top_2_values[2]))

# Keep the remaining parameters fixed at their Task 2.1 Least Squares values
fixed_param_names <- setdiff(names(theta_full), top_2_names)
cat("\nFixed Parameters (held constant at LS estimates):\n")
for (fp in fixed_param_names) {
  cat(sprintf("  %-10s = %10.5f\n", fp, theta_full[fp]))
}
```

```r
# --- Task 3.2: Define Uniform Priors around the estimated values ---
# Set prior ranges around each parameter (+/- 50% width)
delta_1 <- max(0.20, abs(top_2_values[1]) * 0.50)
delta_2 <- max(0.10, abs(top_2_values[2]) * 0.50)

prior_p1_min <- top_2_values[1] - delta_1
prior_p1_max <- top_2_values[1] + delta_1

prior_p2_min <- top_2_values[2] - delta_2
prior_p2_max <- top_2_values[2] + delta_2

cat(sprintf("\nPrior Distributions:\n"))
cat(sprintf("  Prior for %s: Uniform(min = %.4f, max = %.4f)\n", top_2_names[1], prior_p1_min, prior_p1_max))
cat(sprintf("  Prior for %s: Uniform(min = %.4f, max = %.4f)\n", top_2_names[2], prior_p2_min, prior_p2_max))

```

```r
# --- Tasks 3.3 & 3.4: Rejection ABC Simulation ---
# Generate candidate samples from the uniform priors
M_samples <- 100000
set.seed(2026)

cat(sprintf("\nRunning ABC Rejection Sampling with M = %d samples...\n", M_samples))

p1_proposals <- runif(M_samples, min = prior_p1_min, max = prior_p1_max)
p2_proposals <- runif(M_samples, min = prior_p2_min, max = prior_p2_max)

# Compute predictions for all proposed candidate pairs
Phi_full <- candidate_models[[selected_model_name]]

if (length(fixed_param_names) > 0) {
  Phi_fixed         <- Phi_full[, fixed_param_names, drop = FALSE]
  y_fixed_component <- as.vector(Phi_fixed %*% theta_full[fixed_param_names])
} else {
  y_fixed_component <- rep(0, N)
}

y_target_remaining <- y_vec - y_fixed_component
Phi_free           <- Phi_full[, top_2_names, drop = FALSE]

# Vectorized simulation
proposal_matrix  <- rbind(p1_proposals, p2_proposals)
free_predictions <- Phi_free %*% proposal_matrix

# Calculate distance (RMSE) between simulated y and observed y
residuals_matrix <- y_target_remaining - free_predictions
rmse_distances   <- sqrt(colMeans(residuals_matrix^2))

# Accept the top 1% samples with the lowest distance
acceptance_rate   <- 0.01
epsilon_threshold <- quantile(rmse_distances, probs = acceptance_rate)
accepted_idx      <- which(rmse_distances <= epsilon_threshold)
n_accepted        <- length(accepted_idx)

cat(sprintf("Acceptance Threshold (epsilon): %.5f (Top %.1f%%)\n", epsilon_threshold, acceptance_rate * 100))
cat(sprintf("Accepted Posterior Samples:     %d / %d\n", n_accepted, M_samples))

# Extract accepted samples
p1_posterior <- p1_proposals[accepted_idx]
p2_posterior <- p2_proposals[accepted_idx]
post_dist    <- rmse_distances[accepted_idx]

abc_posterior_df <- data.frame(
  p1       = p1_posterior,
  p2       = p2_posterior,
  distance = post_dist
)
names(abc_posterior_df)[1:2] <- top_2_names

```

```r
# --- Task 3.5: Posterior Summary Table & Distribution Plots ---
summarize_posterior <- function(post_samples, prior_min, prior_max, ls_val, param_name) {
  data.frame(
    Parameter   = param_name,
    LS_Estimate = round(ls_val, 5),
    Prior_Min   = round(prior_min, 4),
    Prior_Max   = round(prior_max, 4),
    Post_Mean   = round(mean(post_samples), 5),
    Post_Median = round(median(post_samples), 5),
    Post_StdDev = round(sd(post_samples), 5),
    CI_Lower_95 = round(quantile(post_samples, 0.025), 5),
    CI_Upper_95 = round(quantile(post_samples, 0.975), 5)
  )
}

abc_summary <- rbind(
  summarize_posterior(p1_posterior, prior_p1_min, prior_p1_max, top_2_values[1], top_2_names[1]),
  summarize_posterior(p2_posterior, prior_p2_min, prior_p2_max, top_2_values[2], top_2_names[2])
)

cat("\nABC Posterior Summary Table:\n")
print(abc_summary)
write.csv(abc_summary, file.path(output_dir, "Task3_ABC_Posterior_Summary.csv"), row.names = FALSE)
cat("  Saved: output/Task3_ABC_Posterior_Summary.csv\n")

# Joint posterior distribution plot
p_joint <- ggplot(abc_posterior_df, aes(x = .data[[top_2_names[1]]], y = .data[[top_2_names[2]]])) +
  geom_point(aes(color = distance), alpha = 0.5, size = 1.5) +
  geom_density_2d(color = "black", linewidth = 0.6) +
  annotate("point", x = as.numeric(top_2_values[1]), y = as.numeric(top_2_values[2]),
           color = "red", shape = 3, size = 4, stroke = 2) +
  scale_color_viridis_c(option = "plasma", name = "Distance (RMSE)") +
  labs(title = sprintf("Task 3.4: Joint Posterior Distribution of (%s, %s)", top_2_names[1], top_2_names[2]),
       subtitle = sprintf("Accepted %d samples (tolerance eps = %.4f). Red cross = Least Squares estimate.",
                          n_accepted, epsilon_threshold),
       x = sprintf("Parameter: %s", top_2_names[1]),
       y = sprintf("Parameter: %s", top_2_names[2])) +
  theme_publication()

png(file.path(plots_dir, "Task3_ABC_Joint_Posterior.png"), width = 900, height = 750, res = 120)
print(p_joint)
dev.off()
cat("  Saved: plots/Task3_ABC_Joint_Posterior.png\n")
```

```r
# Marginal posterior distribution plots
p_marg_1 <- ggplot(abc_posterior_df, aes(x = .data[[top_2_names[1]]])) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill = "#4393c3", color = "white", alpha = 0.7) +
  geom_density(color = "#053061", linewidth = 1) +
  geom_vline(xintercept = top_2_values[1], color = "red", linetype = "dashed", linewidth = 1) +
  geom_vline(xintercept = c(abc_summary$CI_Lower_95[1], abc_summary$CI_Upper_95[1]),
             color = "black", linetype = "dotted", linewidth = 0.8) +
  labs(title = sprintf("Marginal Posterior for %s", top_2_names[1]),
       subtitle = sprintf("Mean = %.4f | 95%% CI: [%.4f, %.4f]",
                          abc_summary$Post_Mean[1], abc_summary$CI_Lower_95[1], abc_summary$CI_Upper_95[1]),
       x = top_2_names[1], y = "Posterior Density") +
  theme_publication()

p_marg_2 <- ggplot(abc_posterior_df, aes(x = .data[[top_2_names[2]]])) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill = "#f4a582", color = "white", alpha = 0.7) +
  geom_density(color = "#67001f", linewidth = 1) +
  geom_vline(xintercept = top_2_values[2], color = "red", linetype = "dashed", linewidth = 1) +
  geom_vline(xintercept = c(abc_summary$CI_Lower_95[2], abc_summary$CI_Upper_95[2]),
             color = "black", linetype = "dotted", linewidth = 0.8) +
  labs(title = sprintf("Marginal Posterior for %s", top_2_names[2]),
       subtitle = sprintf("Mean = %.4f | 95%% CI: [%.4f, %.4f]",
                          abc_summary$Post_Mean[2], abc_summary$CI_Lower_95[2], abc_summary$CI_Upper_95[2]),
       x = top_2_names[2], y = "Posterior Density") +
  theme_publication()

png(file.path(plots_dir, "Task3_ABC_Marginal_Posteriors.png"), width = 1100, height = 550, res = 120)
grid.arrange(p_marg_1, p_marg_2, ncol = 2,
             top = grid::textGrob("Task 3.5: Marginal Posterior Distributions with 95% Credible Intervals",
                                  gp = grid::gpar(fontsize = 15, fontface = "bold")))
dev.off()
cat("  Saved: plots/Task3_ABC_Marginal_Posteriors.png\n\n")

cat("================================================================================\n")
cat("Execution complete. All plots and output tables saved successfully.\n")
cat("================================================================================\n")
```